

Incremental View Maintenance over Tree Decompositions as SQL Views: An Evaluation on Feldera

Bachelor's Thesis in Computer Science

submitted by

Ermin Memic

Matriculation number 23-705-197

Department of Informatics
University of Zurich

Data Systems and Theory (DaST) Group
Prof. Dr. Dan Olteanu

Supervisor: Dr. Andrei Draghici

Submission date: 5 August 2026

Abstract

In many settings, a query’s result must stay current as new data arrives. Incremental view maintenance (IVM) provides this by maintaining a structure that encodes the result and updating it from each change, rather than recomputing it from scratch. Abo Khamis et al. give an approach, which we call IVM^+ , that maintains full conjunctive queries in the insert-only setting by organising the computation around a tree decomposition of the query, with a cost governed by the query’s fractional hypertree width. Although the approach is theoretical, in the insert-only setting it can be expressed entirely as a set of SQL views, and so realised on top of an existing incremental engine without modifying it.

This thesis investigates whether that realisation is beneficial in practice. We build a query decomposer that rewrites a join query into the bag views of a tree decomposition, in both IVM^+ and an adjusted variant, together with a semiring extension for computing aggregates such as `COUNT`. We evaluate the rewrites on Feldera, a general-purpose incremental engine, against the baseline of maintaining the query directly, without decomposition, across graph, TPC-H and IMDB workloads.

The rewrite benefits cyclic queries over dense data most, where maintaining the query directly builds large intermediate results. There the decomposition maintains one query 52.11 times faster than the baseline and at lower peak memory, and it renders tractable several queries the baseline cannot finish within a three-hour timeout. A further gain is available whenever only an aggregate of the result is needed, such as its size, rather than the result itself: the semiring count maintains just the bag hierarchy and completes in a fraction of a minute even where the result is far too large for the baseline to materialise, for lower-bound speedups exceeding 1800 times. These gains hold even though the engine does not implement the worst-case-optimal joins the width bound assumes. Where the baseline is already cheap, on acyclic key-based joins and sparse data, the rewrite gives no speedup and adds a memory overhead of up to 2.6 times. Realised as plain SQL views on an unmodified engine, the IVM^+ decomposition is thus a worthwhile rewrite for cyclic queries over dense data, and query-shape-dependent in general.

Contents

1	Introduction	1
2	Background	3
2.1	Data Model and Queries	3
2.2	The Insert-Only Maintenance Problem	3
2.3	Tree Decomposition	4
2.4	Fractional Hypertree Width	5
3	IVM⁺ Approach	6
3.1	Bag Relations and Bridge Variables	6
3.2	IVM ⁺	7
3.3	Variant	7
3.4	Aggregates via Semirings	8
4	Implementation	9
4.1	Feldera	9
4.2	Decomposition as a Feldera Pipeline	10
4.2.1	Tables and connectors	10
4.2.2	Bag views	11
4.2.3	Result reconstruction	11
4.2.4	Computing COUNT(*)	12
4.3	Query Decomposer	12
4.3.1	Parser	12
4.3.2	Hypergraph	13
4.3.3	Decomposition	13
4.3.4	SQL generator	14
4.3.5	Supported queries and limitations	16
4.4	Experiment Harness	16
4.4.1	Orchestrator	17
4.4.2	Execution	17
4.4.3	Result aggregation	18
5	Evaluation	19
5.1	Experimental Setup	19
5.2	Metrics and Pipelines	19
5.2.1	Metrics	19
5.2.2	Pipelines and Variants	20
5.3	Datasets and Queries	21
5.3.1	Graph datasets	21
5.3.2	TPC-H	22
5.3.3	IMDB (JOB)	22
5.4	Hypotheses	22
5.5	Results and Discussion	24
5.5.1	H1: Acyclic Queries	24
5.5.2	H2: Cyclic Queries	25
5.5.3	H3: Reconstruction	26
5.5.4	H4: IVM ⁺ versus the Variant	26

5.5.5	H5: Semiring Extension	27
5.5.6	H6: Orientation	27
5.5.7	Controls	28
6	Conclusion	29
A	Query Definitions	30
A.1	Graph queries	30
A.2	TPC-H queries	30
A.3	IMDB (JOB) queries	31
B	Results Tables	32
B.1	Main experiments	32
B.2	Ablation: terminal-view materialisation	41

Chapter 1

Introduction

Many applications need the result of a query to stay up to date as the underlying database changes. Recomputing that result from scratch after every change is wasteful, since each change usually affects only a small part of it. *Incremental view maintenance* (IVM) instead constructs a data structure that encodes the query result and maintains it incrementally as the database changes, so that the up-to-date result can be read from it when needed [14]. How efficiently a query can be maintained this way depends on its structure and on the type of updates allowed.

Abo Khamis et al. [2] study the maintenance of *full conjunctive queries*, join queries returning all attributes of the relations they combine. We focus on the insert-only setting, in which tuples are only inserted and never deleted. Such workloads arise in many practical settings, such as append-only event streams and continuously growing analytical datasets. They show that in this setting a stream of inserts can be processed one tuple at a time in the same total time as evaluating the query once over the final database. Maintaining the result incrementally is therefore essentially no more costly than computing it once. Their approach organises this maintenance around a *tree decomposition* of the query; we refer to it as the *IVM⁺ approach*.

Although the IVM⁺ approach is theoretical in nature, in the insert-only setting it can be expressed directly as a set of SQL views. The IVM⁺ approach can therefore be realised on top of an existing IVM engine simply by rewriting the input query into these views, without modifying the engine itself. This makes it lightweight to deploy, requiring only a query rewrite rather than the design or modification of a dedicated system.

In this thesis, we investigate whether the IVM⁺ approach is beneficial in practice when realised at the SQL level on an existing IVM engine. We make two contributions:¹

- **A query decomposer.** We present a tool that, given a join query, computes a tree decomposition and produces the corresponding IVM⁺ realisation as a set of SQL views. The entire translation is performed at the SQL level, without modifying the engine. The tool supports both the standard IVM⁺ approach and an adjusted variant that we introduce in Chapter 3. It also supports aggregates such as `COUNT`, based on the concept of provenance semirings [9], for both the standard approach and the variant.
- **An empirical evaluation on Feldera.** As our target engine we use Feldera [7], a general-purpose engine based on the DBSP model [5] that incrementally maintains SQL views as the underlying data changes. In Feldera, queries are expressed as SQL views, which makes the rewritten IVM⁺ views a natural fit. We compare the maintenance performance of the rewritten queries against that of the original queries.

Our evaluation finds that the rewrite helps most where the original query would otherwise build large intermediate results, on cyclic queries over dense data. There it maintains the result far more efficiently, in one case 52.11 times faster than the original query and using less memory, and it keeps tractable several queries the original cannot finish within three hours. When only the size of the result is needed rather than the result itself, the gain is larger still, completing in a fraction of a minute even where the original query cannot finish, for speedups of over 1800 times. Where the original query is

¹The thesis repository is available at <https://github.com/ermin-mumic/UZH-Bachelor-Thesis-IVM>.

already cheap to maintain, the rewrite gives no speedup and uses up to 2.6 times the memory, so its benefit is specific to these harder, cyclic queries rather than a property of the rewrite in general.

The remainder of this thesis is organised as follows. Chapter 2 introduces the necessary background: the query model, the insert-only maintenance problem, tree decompositions and fractional hypertree width. Chapter 3 presents the IVM⁺ approach, our adjusted variant, and a semiring extension for computing aggregates. Chapter 4 describes the query decomposer and the experiment harness, and how the approach is realised as Feldera views. Chapter 5 reports the experimental setup, datasets, and results. Chapter 6 concludes and discusses future work.

Chapter 2

Background

This chapter introduces the formal foundations underlying the IVM⁺ approach and the adjusted variant studied in this thesis. We fix our data model and query class and introduce a running example we revisit throughout, state the insert-only maintenance problem the approach addresses, and present tree decompositions and the fractional hypertree width derived from them.

2.1 Data Model and Queries

We use standard relational terminology, which we briefly recall here [1].

A *relation* R is a finite set of tuples, and a *database* $\mathcal{D}$ is a finite set of relations. The *size* $|R|$ of a relation is the number of its tuples, and the size $|\mathcal{D}|$ of a database is the sum of the sizes of its relations.

A *full conjunctive query*, also called a *natural join query* [2], has the form

$$Q = R_1(\mathbf{X}_1) \bowtie \cdots \bowtie R_k(\mathbf{X}_k),$$

where R_i is a relation symbol and $\mathbf{X}_i$ is a tuple of variables, the *schema* of R_i . Each $R_i(\mathbf{X}_i)$ is an *atom* of Q , and $\text{vars}(Q) = \bigcup_{i=1}^k \mathbf{X}_i$ denotes the set of all variables of Q . We call such a query *full* because every variable in $\text{vars}(Q)$ is returned in the result; no variables are projected away. For brevity we refer to full conjunctive queries simply as *queries*.

A tuple t over $\text{vars}(Q)$ is in the *result* $Q(\mathcal{D})$ if each atom $R_i(\mathbf{X}_i)$ can be matched to a tuple $t_i \in R_i$ so that the matched tuples agree on their shared variables: for every variable $x \in \text{vars}(Q)$, all matched tuples t_i with $x \in \mathbf{X}_i$ assign x the same value, namely $t(x)$. For a tuple t and a variable x in its schema, $t(x)$ denotes the value that t assigns to x .

Running example. As a running example we use a small query over a single binary relation E , which we read as the edge set of a directed graph. Consider

$$Q_\diamond = E_1(A, B) \bowtie E_2(B, C) \bowtie E_3(C, D) \bowtie E_4(B, D) \bowtie E_5(A, D),$$

with $\text{vars}(Q_\diamond) = \{A, B, C, D\}$. Here $E_1, \dots, E_5$ all denote the same relation E ; we treat each atom as a distinct occurrence of E over the same data, which is standard and does not affect the tree decompositions and widths defined in the following sections. We examine its structure and tree decomposition in Section 2.3 and determine its fractional hypertree width in Section 2.4; we return to $Q_\diamond$ throughout the thesis, including as a benchmark query in our experiments (Chapter 5).

2.2 The Insert-Only Maintenance Problem

We study the *insert-only* incremental view maintenance problem for a fixed query Q [2]. The database $\mathcal{D}$ is initially empty and grows by single-tuple insertions, applied one at a time, and neither their number nor which tuples they insert is known in advance. Writing $\mathcal{D}^{(0)} = \emptyset$ and $\mathcal{D}^{(\tau)}$ for the database after the τ -th insertion, the task is to maintain the result $Q(\mathcal{D}^{(\tau)})$ at each time-step τ . The cost of interest is the time needed to process the insertions.

Abo Khamis et al. also study the *insert-delete* setting, in which tuples may additionally be deleted, and the maintenance of the *change* to the result rather than the full result. We consider only insert-only maintenance of the full result.

2.3 Tree Decomposition

The structure of a query is captured by its *hypergraph*: the vertices are the variables $\text{vars}(Q)$, and each atom $R_i(\mathbf{X}_i)$ contributes a hyperedge on its variables $\mathbf{X}_i$. When all atoms are binary, as in our running example, the hypergraph is an ordinary undirected graph. For $Q_\diamond$ it is the *diamond* shown in Figure 2.1: two triangles $\{A, B, D\}$ and $\{B, C, D\}$ sharing the edge $B-D$.

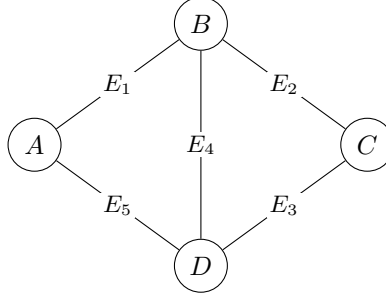


Figure 2.1: The hypergraph of $Q_\diamond$.

A *tree decomposition* [6] $(\mathcal{T}, \chi)$ of a query $Q = R_1(\mathbf{X}_1) \bowtie \dots \bowtie R_k(\mathbf{X}_k)$ consists of a rooted tree $\mathcal{T}$, whose nodes are called *bags*, together with a function χ that assigns each bag b a set of variables $\chi(b)$. A valid tree decomposition satisfies two conditions:

1. **Coverage:** Every atom $R_i(\mathbf{X}_i)$ is assigned to at least one bag b with $\mathbf{X}_i \subseteq \chi(b)$.
2. **Connectivity:** For every variable $x \in \text{vars}(Q)$, the set of bags $\{b \mid x \in \chi(b)\}$ forms a connected subtree of $\mathcal{T}$.

Running example. Figure 2.2 shows a tree decomposition of $Q_\diamond$. It has two bags, one for each triangle of the hypergraph (Figure 2.1):

- the root bag $b_1 = \{A, B, D\}$, covering the atoms $E_1(A, B)$, $E_4(B, D)$, and $E_5(A, D)$;
- the child bag $b_2 = \{B, C, D\}$, covering the atoms $E_2(B, C)$, $E_3(C, D)$, and $E_4(B, D)$.

Both conditions hold: every atom is covered, with the shared atom $E_4(B, D)$ lying in both bags, and every variable induces a connected subtree, since A and C each occur in a single bag while B and D occur in the two adjacent bags. This is one of several valid tree decompositions of $Q_\diamond$; Section 2.4 makes precise how their quality is measured.

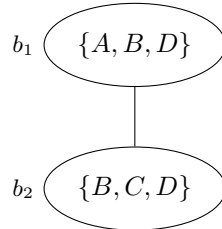


Figure 2.2: A tree decomposition of $Q_\diamond$.

2.4 Fractional Hypertree Width

Let $\mathbf{V} \subseteq \text{vars}(Q)$ be a set of variables. A *fractional edge cover* [3] of $\mathbf{V}$ assigns a weight $\lambda_i \in [0, 1]$ to each atom $R_i(\mathbf{X}_i)$ of Q such that, for every variable $x \in \mathbf{V}$, the weights of the atoms whose schema contains x sum to at least one. The *fractional edge cover number* $\rho^*(\mathbf{V})$ is the smallest total weight of such a cover:

$$\begin{aligned} \min \quad & \sum_{i=1}^k \lambda_i \\ \text{s.t.} \quad & \sum_{i: x \in \mathbf{X}_i} \lambda_i \geq 1 \quad \forall x \in \mathbf{V}, \\ & \lambda_i \in [0, 1] \quad \forall i \end{aligned}$$

The *fractional hypertree width* [10] $w(Q)$ of Q is the smallest value, over all tree decompositions $(\mathcal{T}, \chi)$, of the largest fractional edge cover number $\rho^*(\chi(b))$ among the bags b of $\mathcal{T}$:

$$w(Q) = \min_{(\mathcal{T}, \chi)} \max_{b \in \mathcal{T}} \rho^*(\chi(b)).$$

In particular, a tree decomposition with a single bag has that bag equal to $\text{vars}(Q)$, so the maximum over its bags is just $\rho^*(\text{vars}(Q))$; since $w(Q)$ minimises over all tree decompositions, $w(Q) \leq \rho^*(\text{vars}(Q))$.

Running example. The single-bag decomposition has $\rho^*(\{A, B, C, D\}) = 2$: giving $E_1(A, B)$ and $E_3(C, D)$ weight 1 each covers all four variables, and no cover is lighter. The two-triangle decomposition of Figure 2.2 does better: in each triangle bag, weight $\frac{1}{2}$ on each of its three edges covers its three variables, so $\rho^*(\chi(b)) = \frac{3}{2}$ for both bags. No tree decomposition can do better, hence $w(Q_\diamond) = \frac{3}{2} < 2$.

The IVM^+ approach is built on a tree decomposition of minimum fractional hypertree width, which governs the cost of maintaining the query [2]. Concretely, when each bag is evaluated with a worst-case-optimal join, a sequence of N insertions is maintained in total time $O(N^{w(Q)})$. Acyclic queries have $w(Q) = 1$, since every atom can occupy its own bag, whereas a cycle forces some bag to cover more than one atom, giving $w(Q) > 1$. Computing such a decomposition is NP-hard [8], and approximation algorithms exist [13]. Our decomposer instead uses a practical heuristic: it generates candidate decompositions with a Minimum Fill-in heuristic [4] and selects the one of least fractional hypertree width, as described in more detail in Chapter 4.

Chapter 3

IVM⁺ Approach

This chapter presents the IVM⁺ approach of Abo Khamis et al. [2], an adjusted variant that we introduce, and a semiring-based extension for computing aggregates. Throughout, the approach is described at the level of relational views, independently of any particular engine, and its realisation on Feldera is deferred to Chapter 4.

3.1 Bag Relations and Bridge Variables

Both IVM⁺ and its variant are built on a fixed tree decomposition $(\mathcal{T}, \chi)$ of the query Q (Section 2.3): a rooted tree $\mathcal{T}$ together with a function χ assigning each bag b a set of variables $\chi(b)$. Following Chapter 2, we use a decomposition of minimum fractional hypertree width.

For each bag b the approach maintains a Q' -relation Q'_b and a P' -relation P'_b . These definitions are shared by IVM⁺ and its variant, which differ only in the underlying Q -relation Q_b (Sections 3.2 and 3.3). Defining the Q' - and P' -relations requires one further notion, introduced next: the *bridge variables* of a bag.

For every non-root bag b , the *bridge variables* $\mathbf{Y}_b$ are the variables it shares with its parent:

$$\mathbf{Y}_b = \chi(b) \cap \chi(\text{parent}(b)).$$

The Q' - and P' -relations are computed bottom-up, from the leaves to the root. For each bag b , the Q' -relation extends the bag's own Q -relation with the P' -relations of its children, and the P' -relation projects the result onto the bridge variables:

$$Q'_b(\chi(b)) = Q_b(\chi(b)) \bowtie \bigwedge_{c \in \text{children}(b)} P'_c(\mathbf{Y}_c), \quad (3.1)$$

$$P'_b(\mathbf{Y}_b) = \pi_{\mathbf{Y}_b} Q'_b(\chi(b)). \quad (3.2)$$

Each child's P' -relation acts as a *semijoin filter* on its parent's Q' -relation: it retains only those tuples of Q'_b that agree, on the bridge variables $\mathbf{Y}_c$, with some tuple of the child's Q'_c . Because each Q'_c is itself filtered by its own children, a surviving tuple of Q'_b has a matching tuple in each of the subtrees beneath b . A leaf bag has no children, so $Q'_b = Q_b$. Once all children of a bag b have their P' -relations, Q'_b and then P'_b can be computed.

The full query result is obtained by joining all the Q' -relations along their shared variables:

$$Q(\mathcal{D}) = \bigwedge_{b \in \mathcal{T}} Q'_b.$$

Joining the plain Q -relations instead yields the same result, since the P' -semijoins only discard tuples that would not survive the reconstruction join in any case.

Running example. For $Q_\diamond$ and the decomposition of Figure 2.2, the child bag $b_2 = \{B, C, D\}$ shares $\{B, D\}$ with the root $b_1 = \{A, B, D\}$, so its bridge variables are $\mathbf{Y}_{b_2} = \{B, D\}$, while the root has none. Since b_2 is a leaf, its Q' -relation is simply $Q'_{b_2} = Q_{b_2}$. It passes the projection P'_{b_2} up to the root, which combines it with Q_{b_1} to form Q'_{b_1} , as shown in Figure 3.1. The full result is then $Q(\mathcal{D}) = Q'_{b_1} \bowtie Q'_{b_2}$.

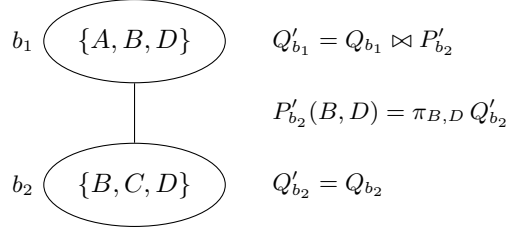


Figure 3.1: Bag relations and bridge variables on the tree decomposition of $Q_\diamond$.

3.2 IVM⁺

The IVM⁺ approach completes the construction of Section 3.1 by defining each bag's Q -relation as the *restriction* of the whole query to the bag's variables, a notion we make precise first.

Given a query $Q = R_1(\mathbf{X}_1) \bowtie \dots \bowtie R_k(\mathbf{X}_k)$ and a set of variables $\mathbf{V}$, the *restriction* $Q_{\mathbf{V}}$ projects each atom onto its variables in $\mathbf{V}$, keeping only the atoms that share at least one variable with $\mathbf{V}$:

$$Q_{\mathbf{V}} = \bigbowtie_{i: \mathbf{X}_i \cap \mathbf{V} \neq \emptyset} \pi_{\mathbf{X}_i \cap \mathbf{V}} R_i(\mathbf{X}_i).$$

The IVM⁺ approach then defines the Q -relation of each bag b as the restriction of Q to the bag's variables:

$$Q_b = Q_{\chi(b)}. \quad (3.3)$$

An atom whose variables lie entirely within $\chi(b)$ is kept in full, while an atom that only partially overlaps $\chi(b)$ contributes its projection onto the shared variables. Like the child P' -relations in (3.1)–(3.2), the projected atoms act as *semijoin filters*: they discard tuples of Q_b that would not be part of any query result anyway, aiming to keep it small.

Running example. For $Q_\diamond$ and the decomposition of Figure 3.1, which also shows the bridge variables and the resulting Q' - and P' -relations, it remains to give the underlying Q -relations. The root bag $b_1 = \{A, B, D\}$ contains $E_1(A, B)$, $E_4(B, D)$, and $E_5(A, D)$ in full, whereas $E_2(B, C)$ and $E_3(C, D)$ overlap it only in B and D and are projected accordingly:

$$Q_{b_1} = E_1(A, B) \bowtie E_4(B, D) \bowtie E_5(A, D) \bowtie \pi_B E_2(B, C) \bowtie \pi_D E_3(C, D).$$

Symmetrically, the child bag $b_2 = \{B, C, D\}$ keeps $E_2(B, C)$, $E_3(C, D)$, and $E_4(B, D)$ in full and projects the other two atoms:

$$Q_{b_2} = E_2(B, C) \bowtie E_3(C, D) \bowtie E_4(B, D) \bowtie \pi_B E_1(A, B) \bowtie \pi_D E_5(A, D).$$

The Q' - and P' -relations then follow from (3.1)–(3.2) as before, now with these concrete Q -relations (Figure 3.1).

3.3 Variant

Recall that the IVM⁺ approach sets each bag's Q -relation to the restriction $Q_b = Q_{\chi(b)}$ (3.3), keeping every atom that shares at least one variable with $\chi(b)$ and projecting the partially overlapping ones onto the bag's variables. We investigate a *variant* that adjusts this one definition: it keeps only the atoms whose variables are *fully contained* in the bag, with no projections. Formally, an atom $R_i(\mathbf{X}_i)$ enters Q_b^* exactly when $\mathbf{X}_i \subseteq \chi(b)$:

$$Q_b^* = \bigbowtie_{i: \mathbf{X}_i \subseteq \chi(b)} R_i(\mathbf{X}_i). \quad (3.4)$$

An atom that only partially overlaps $\chi(b)$, i.e. with $\mathbf{X}_i \not\subseteq \chi(b)$, is dropped entirely rather than projected. The Q' - and P' -relations are built exactly as before, via (3.1)–(3.2), now on Q_b^* in place of Q_b .

A projected atom can play two roles. Sometimes it genuinely filters Q_b , removing tuples that will not contribute to the result. Other times it is redundant, because the child P' -relations that

Q'_b joins with already remove the same tuples. The variant drops the projected atoms: where they were redundant this removes wasted join work, and where they were doing real filtering it lets larger intermediate relations through. Which effect wins depends on the query and the data, an empirical question we take up in Chapter 5.

Dropping the projected atoms does not change the result. By coverage, every atom is fully contained in at least one bag, so it appears in full in that bag's Q^* -relation. The reconstruction join of the Q^* -relations therefore yields the join of all atoms, which is $Q(\mathcal{D})$. As in Section 3.1, joining the Q' -relations instead gives the same result. The dropped projections only duplicated atoms present in full elsewhere, so they carried no constraint. The variant therefore differs from the standard approach in intermediate size and join work, not in its result.

Running example. For $Q_\diamond$ and the decomposition of Figure 3.1, the root bag $b_1 = \{A, B, D\}$ keeps only the atoms contained in $\{A, B, D\}$, dropping the projections that the standard Q_{b_1} carried:

$$Q_{b_1}^* = E_1(A, B) \bowtie E_4(B, D) \bowtie E_5(A, D).$$

Likewise the child bag $b_2 = \{B, C, D\}$ keeps only its contained atoms:

$$Q_{b_2}^* = E_2(B, C) \bowtie E_3(C, D) \bowtie E_4(B, D).$$

Compared with Q_{b_1} and Q_{b_2} in Section 3.2, the projected atoms $\pi_B E_2, \pi_D E_3$ and $\pi_B E_1, \pi_D E_5$ are gone. The Q' - and P' -relations are then formed from these via (3.1)–(3.2) as usual.

3.4 Aggregates via Semirings

So far, the full query result is obtained by joining the Q' -relations of all the bags in the tree decomposition (Section 3.1). In many cases, however, one does not need to enumerate the full result, only an *aggregate* over it, such as the number of result tuples counted by `COUNT(*)`. Such an aggregate can be computed directly on the decomposition, without ever materialising the full result, by attaching *semiring annotations* to the tuples of the atoms.

A semiring annotation tags each tuple with a value from a semiring domain that the query carries along and combines as it joins and projects. By choosing what these values are and how they combine, the same query computes different summaries of the result, such as counts, probabilities, or provenance [9]. We use this to compute `COUNT(*)`. Its realisation as SQL views, for both IVM^+ and the variant, is described in Chapter 4, and its performance is evaluated in Chapter 5.

Chapter 4

Implementation

Chapter 3 described the IVM^+ approach and its variant at the level of relational algebra, independently of any particular engine. This chapter realises them on Feldera and presents the two tools we built: a *query decomposer* and an *experiment harness*.

The two share a single artifact, the set of *SQL views* that encode a decomposition. The query decomposer produces these views from an input query, and the experiment harness deploys them as a Feldera pipeline, runs it over the input data, and measures performance. Figure 4.1 shows the two components.

The SQL views used in our evaluation were written by hand rather than produced by the decomposer, which we developed afterwards to generate them automatically. They are semantically equivalent to its output and differ only in syntax.

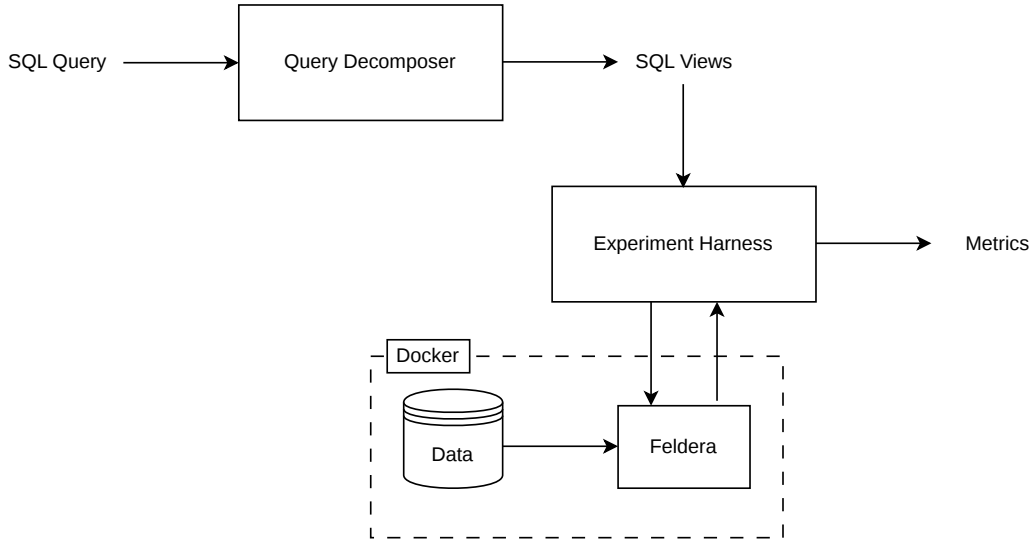


Figure 4.1: Overview of the query decomposer and the experiment harness.

We first give the necessary background on how Feldera works, then show how the decomposition of Chapter 3 becomes a Feldera pipeline, and finally describe the decomposer and the harness in turn.

4.1 Feldera

Feldera is a streaming query engine for *incremental computation* [7]. A computation is expressed in SQL, and Feldera maintains its results as the input data changes, updating them from the changes alone rather than recomputing over all the data. This incremental evaluation is founded on the DBSP model [5] and applies to arbitrary SQL queries.

A *Feldera pipeline* is a set of SQL tables and views [7]. The tables are the inputs, and the views are derived from them and can be nested within one another. As changes arrive at the tables, Feldera

updates every view accordingly. This maps directly onto the construction of Chapter 3: the bag relations become SQL views, defined bottom-up from the base tables and the child bags' views, so the IVM⁺ approach and its variant run as an ordinary Feldera pipeline on an unmodified engine.

Changes reach a pipeline's tables in three ways: through *input connectors* attached to the tables, through HTTP requests, or through ad-hoc `INSERT` statements [7]. Our experiments use input connectors, which read the input data from CSV files. We chose them over HTTP ingress because posting the data over HTTP could not keep the pipeline fed: even when sent in batches, each batch was processed almost instantly and the pipeline then idled until the next arrived, so a run measured how fast data could be delivered rather than how fast Feldera maintained the views. Reading directly from local CSV files through the connectors keeps the engine saturated, so that the incremental computation is what is measured.

By default, Feldera stores only the data needed to compute future outputs, not the complete contents of every table and view [7]. A table or view can be declared *materialised* to keep its full contents available, for example to inspect or query them directly. A table with a `PRIMARY KEY` is materialised automatically, since Feldera must maintain a key index. Materialising a table or view can slow the pipeline down, as more data must be written to storage, which can also increase the pipeline's memory use.

4.2 Decomposition as a Feldera Pipeline

This section realises the construction of Chapter 3 as a Feldera pipeline. Each bag becomes a Q' -view for its Q' -relation, and each non-root bag additionally a P' -view for its P' -relation. The views are built bottom-up, so a child's P' -view is ready when its parent's Q' -view is computed, and a final `RESULT` view produces the query's output. We name a non-root bag's views `BAG_N_QPRIME` and `BAG_N_PPRIME`, and the root's `ROOT_QPRIME`. IVM⁺ and the variant differ only in how the Q' -view is defined. The view definitions are ordinary SQL, and only the input connectors and the materialisation annotations on the tables are specific to Feldera.

4.2.1 Tables and connectors

The base tables carry a Feldera *input connector* that specifies where the pipeline reads its data. For $Q_\circ$, whose five atoms are all occurrences of one edge relation, this is a single table `EDGES(src, tgt)` with a connector that reads a CSV file:

Listing 4.1: The `EDGES` table with a CSV input connector.

```
CREATE TABLE EDGES (
  src BIGINT,
  tgt BIGINT
) WITH (
  'materialized' = 'true',
  'connectors' = '[{
    "transport": {
      "name": "file_input",
      "config": {
        "path": "edges.csv"
      }
    },
    "format": {
      "name": "csv",
      "config": {
        "headers": true
      }
    }
  }]'
);
```

Besides the connector, the table is declared `'materialized'`, the annotation from Section 4.1 that keeps its full contents available. The connector and this annotation are the only Feldera-specific parts of the declaration.

4.2.2 Bag views

A bag's P' -view projects its Q' -view onto the bridge variables, giving the set of bridge values with which the parent filters its own tuples (Chapter 3). Here SQL's bag semantics matter: a plain projection keeps duplicate rows, but the P' -relation must be a *set*, or the duplicates would multiply the parent's tuples when it joins with the view. We therefore use `SELECT DISTINCT`. For $Q_\diamond$, the child bag $b_2 = \{B, C, D\}$ shares $\{B, D\}$ with the root, so its P' -view is:

Listing 4.2: The P' -view of the child bag b_2 .

```
CREATE MATERIALIZED VIEW BAG_2_PPRIME AS
SELECT DISTINCT B, D
FROM BAG_2_QPRIME;
```

The view is materialised and can therefore be queried directly (Section 4.1). The Q' -view `BAG_2-QPRIME` it reads from is defined next.

We define a bag's Q' -view by joining its atoms on their shared variables with `USING`. Unlike an `ON` join, `USING` merges each pair of shared columns into one, so `SELECT *` returns exactly the bag's variables and the views chain cleanly. Because $Q_\diamond$ is a self-join, with every atom an occurrence of the same `EDGES` relation, each atom is written as a subquery that renames `src` and `tgt` to that atom's variables. For IVM^+ , a partially overlapping atom is added as an inline `SELECT DISTINCT` semijoin. The child bag b_2 is a leaf, so its Q' -view is just its atoms:

Listing 4.3: The Q' -view of the child bag b_2 (IVM^+).

```
CREATE MATERIALIZED VIEW BAG_2_QPRIME AS
SELECT *
FROM (SELECT src AS B, tgt AS C FROM EDGES)
JOIN (SELECT src AS C, tgt AS D FROM EDGES) USING (C)
JOIN (SELECT src AS B, tgt AS D FROM EDGES) USING (B, D)
JOIN (SELECT DISTINCT tgt AS B FROM EDGES) USING (B)
JOIN (SELECT DISTINCT tgt AS D FROM EDGES) USING (D);
```

The first three subqueries are the contained atoms $E_2(B, C)$, $E_3(C, D)$, and $E_4(B, D)$, joined on their shared variables. The last two are the semijoins $\pi_B E_1$ and $\pi_D E_5$, restricting B and D to values that occur in those atoms. The variant keeps only the fully contained atoms and drops the two semijoins:

Listing 4.4: The Q' -view of the child bag b_2 (variant).

```
CREATE MATERIALIZED VIEW BAG_2_QPRIME AS
SELECT *
FROM (SELECT src AS B, tgt AS C FROM EDGES)
JOIN (SELECT src AS C, tgt AS D FROM EDGES) USING (C)
JOIN (SELECT src AS B, tgt AS D FROM EDGES) USING (B, D);
```

A non-leaf bag is built the same way but also joins its children's P' -views on the bridge variables. The root's Q' -view `ROOT_QPRIME`, for example, joins `BAG_2_PPRIME` using $\{B, D\}$.

4.2.3 Result reconstruction

The full query result is reconstructed by joining all the Q' -views on their shared variables (Chapter 3). For $Q_\diamond$ this is a single join of the root and child Q' -views on the bridge variables $\{B, D\}$:

Listing 4.5: The `RESULT` view reconstructing the full output.

```
CREATE MATERIALIZED VIEW RESULT AS
SELECT *
FROM ROOT_QPRIME
JOIN BAG_2_QPRIME USING (B, D);
```

This `RESULT` view is the terminal view an experiment measures. We also consider a variant *without reconstruction* that omits this final join: the root's Q' -view `ROOT_QPRIME` is the terminal view instead. This isolates the cost of maintaining the bag views from the cost of joining them back into the full result. For aggregate queries the terminal view is neither of these, but the aggregate itself, computed as described next.

4.2.4 Computing COUNT(*)

The number of result tuples returned by COUNT(*) can be computed directly on the decomposition, without materialising the full result (Section 3.4). Each P' -view then carries a *count* beside its bridge variables: instead of SELECT DISTINCT, it groups by the bridge variables and records how many tuples fall in each group. A leaf bag’s count is COUNT(*): every tuple has an implicit count of one, so counting the rows in each group gives its count, with no explicit sum. A bag with children instead multiplies its children’s counts and sums the products within each group. For $Q_\diamond$, the child bag b_2 is a leaf, so its P' -view groups by $\{B, D\}$ and counts:

Listing 4.6: The P' -view of the child bag, carrying a count.

```
CREATE MATERIALIZED VIEW BAG_2_PPRIME AS
SELECT B, D, COUNT(*) AS ANNOTATION_2
FROM BAG_2_QPRIME
GROUP BY B, D;
```

The root’s Q' -view joins this P' -view as before, and so carries the column ANNOTATION_2. The RESULT view is then the sum over the root’s Q' -view of the counts contributed by its child:

Listing 4.7: The RESULT view computing COUNT(*) .

```
CREATE MATERIALIZED VIEW RESULT AS
SELECT SUM(ANNOTATION_2) AS RESULT
FROM ROOT_QPRIME;
```

This RESULT is the number of tuples in the result of $Q_\diamond$, which is its COUNT(*). Because every input tuple has an implicit annotation of one, the base tables need no annotation column, and an atom appearing in several bags is not double-counted, since multiplying by one changes nothing. The same semiring construction handles other aggregates. One that reads a column, such as MIN, carries its value explicitly and combines it in a single *canonical* bag so that it is not aggregated more than once. We implement COUNT(*) as a representative case.

Both the reconstruction and the COUNT(*) above assume the base relations are sets, as Section 2.1 defines a relation to be. The reconstruction relies on the natural join being idempotent, $R \bowtie R = R$: an atom contained in several bags is effectively joined with itself when their Q' -views are recombined, which changes nothing on a set. The count relies on the same fact through the implicit annotation of one. Under SQL’s bag semantics this no longer holds: a duplicated input row makes such an atom count once per bag it lies in, so the result over-counts. Our datasets, introduced in Chapter 5, are set-valued (the relational ones are primary-keyed and the graph edge sets carry no duplicate edges), so this does not arise. It could also be handled directly by counting each atom in a single *canonical* bag and joining a deduplicated projection of it elsewhere, the same mechanism used for MIN above.

4.3 Query Decomposer

Section 4.2 built the SQL views for $Q_\diamond$ by hand. The *query decomposer* automates this: given a query, it computes a tree decomposition and emits the corresponding Q' - and P' -views, in both the IVM⁺ and the variant form. It is the tool one would run to obtain a pipeline for a new query, without working through the decomposition by hand.

The decomposer takes two inputs, the query and the schema of the tables it refers to, and returns the set of SQL views. It proceeds in four stages, shown in Figure 4.2: a *parser* reads the SQL, a *hypergraph* is built from the query, a *tree decomposition* is computed over it, and a *generator* emits the views. We describe each stage in turn.

4.3.1 Parser

The parser reads the two SQL inputs, the schema and the query, and extracts the structure the later stages work on. It needs the full schema, not just the query, for two reasons. First, the tool returns the full conjunctive query, so it must know every attribute of every relation, including those the query never mentions. Second, the hypergraph has one vertex per variable, so all variables must be known even where the query does not reference them. Parsing itself uses *sqlglot*, a Python SQL parsing library, which turns the SQL into an abstract syntax tree that we then traverse.

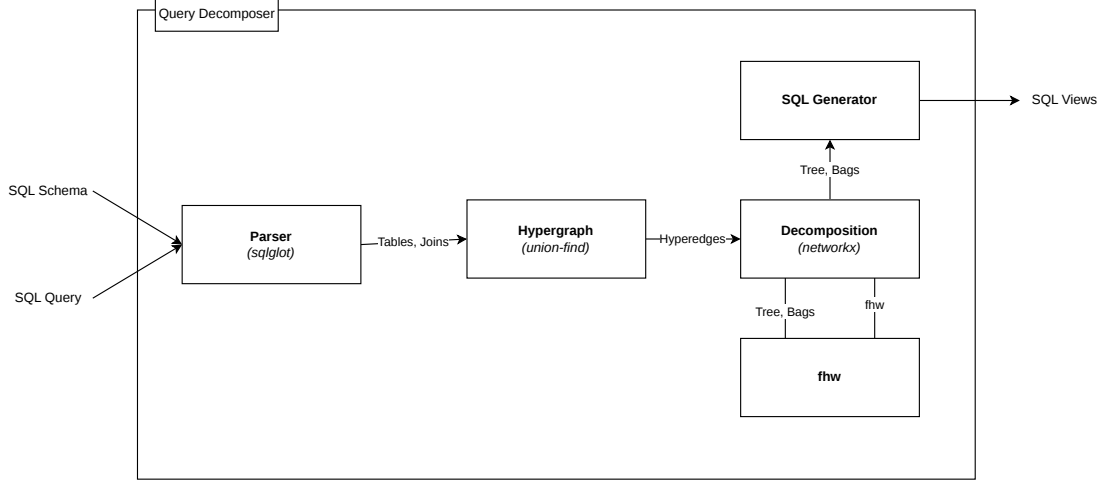


Figure 4.2: The stages of the query decomposer.

The parser traverses the syntax tree clause by clause. It first reads the **SELECT** clause, which fixes whether the query asks for the full result or a **COUNT(*)**. It then walks the **FROM** clause and each **JOIN** clause in turn, and as it reaches a table it registers it, resolving its alias and recording its variables. SQL lets a query rename tables and columns and use the same relation more than once, so a self-join, where one relation appears under several aliases, is registered as several independent relations, matching the atoms of Section 2.1: the five occurrences of **EDGES** in $Q_\diamond$ become five relations $E_1, \dots, E_5$. In the same pass it reads each join’s equi-join condition, written as **JOIN ... ON** or **JOIN ... USING**, telling the next stage which columns are equated and therefore collapse into a single vertex of the hypergraph, even where those columns have different names. Finally it reads the **WHERE** clause, collecting any single-relation filters. The relations, their variables, the join conditions, and the single-relation filters are then passed on to the later stages. The queries the parser accepts, and what it rejects, are set out in Section 4.3.5.

4.3.2 Hypergraph

The hypergraph stage builds the query’s hypergraph, defined in Section 2.3: its vertices are the query’s variables and each relation contributes one hyperedge. A *column* is an attribute of a relation in the SQL text, while a *variable* is a vertex of the hypergraph, and a single variable can stand for several columns. As noted above, an equi-join can equate two columns that do not share a name, and equated columns denote the same variable, so they must collapse into one. The stage computes this with a *union-find* structure: every column starts in its own group, each join condition merges the groups of the two columns it equates, and each final group is one variable. Merging groups rather than pairs handles chains of joins, where equating $A = B$ and $B = C$ places all three columns in one variable. Each variable takes its name from one of its columns, so two different variables would clash when their columns happen to share a name although no join equates them. The stage renames such variables to keep every name distinct. It also records, in both directions, which column of each relation a variable stands for, since the generator later needs to recover the physical column behind each variable. Each relation then contributes one hyperedge, the set of variables of its columns. For $Q_\diamond$ this yields the four variables A, B, C, D and five hyperedges, the diamond of Figure 2.1, which is passed to the decomposition stage.

4.3.3 Decomposition

Section 2.4 noted that computing a tree decomposition of minimum fractional hypertree width is NP-hard, so the decomposer uses a heuristic instead. It relies on the Minimum Fill-in heuristic [4] from the *networkx* library, which operates on ordinary graphs rather than hypergraphs, so the stage first builds the *primal graph* of the hypergraph: each hyperedge, that is each relation’s set of variables, becomes a clique. Making every hyperedge a clique forces its variables into a common bag, so a tree decomposition of the primal graph is a tree decomposition of the hypergraph [10].

Minimum Fill-in is a vertex-elimination heuristic for finding a decomposition of small treewidth [4]. It repeatedly eliminates the vertex whose neighbourhood is closest to a clique, measured by its number of *fill-in edges*, the edges missing among its neighbours that would make them pairwise adjacent. A vertex whose neighbours are already pairwise adjacent has no fill-in. To eliminate a vertex the heuristic adds those missing edges, making its neighbours a clique, records the vertex together with its neighbours as a bag, and removes the vertex and its incident edges. From these bags networkx forms the tree. The stage then removes any bag contained in a neighbouring bag, so that none is redundant.

A single run of Minimum Fill-in produces one decomposition. When several vertices tie for the fewest fill-in edges, the networkx implementation breaks the tie by the order in which the vertices were inserted into the primal graph. The decomposer exploits this to obtain several candidates: it runs Minimum Fill-in once for each of a number of insertion orderings, a sorted baseline and twenty seeded shuffles, so that different tie-breaks yield different decompositions.

Minimum Fill-in targets small treewidth, but the IVM⁺ approach wants small fractional hypertree width (Section 2.4). The two are related: the fractional hypertree width of a query is at most its treewidth plus one [10], so decompositions of small treewidth are a good source of ones of small fractional hypertree width. But two decompositions of equal treewidth can still differ in fractional hypertree width, so the decomposer computes it for each candidate and keeps the smallest, breaking ties by treewidth. The fractional hypertree width of a candidate is the largest fractional edge cover number over its bags, obtained by solving the linear program of Section 2.4 with *scipy*. For $Q_\diamond$ this selects the two-triangle decomposition of Figure 2.2, of fractional hypertree width $\frac{3}{2}$. The selected bags and tree are passed to the SQL generator.

4.3.4 SQL generator

The generator turns the bags and tree from the previous stage into the SQL views of Section 4.2, producing automatically the translation presented there for $Q_\diamond$. It first *orients* the tree by choosing a root, by default bag 1, though any bag may serve as root, arranging the bags into a rooted tree. For each non-root bag it then computes the *bridge variables* it shares with its parent.

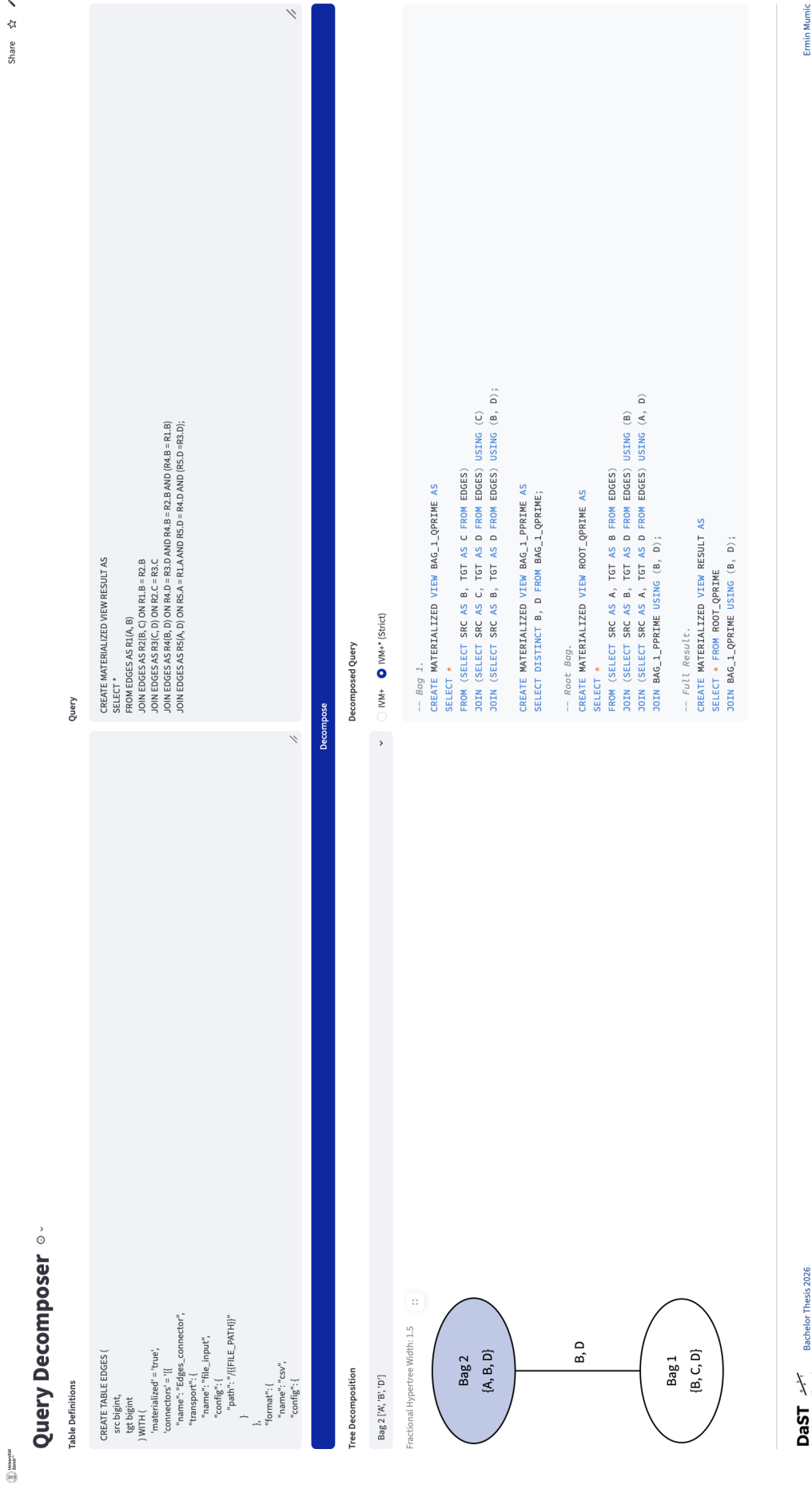
The views are emitted bottom-up, each bag after its children, so a child's P' -view is ready when its parent needs it, matching the construction of Section 4.2. For a bag, the generator builds its Q' -view from the atoms. Each atom becomes a subquery over its underlying table: using the mapping the hypergraph stage recorded, the generator selects the physical column each of the bag's variables stands for and aliases it to the variable, as in the `src AS B` of Section 4.2, and appends the atom's single-relation filters as a `WHERE` clause. The subqueries are joined with `JOIN ... USING` on their shared variables. IVM⁺ and the variant differ only here, in that IVM⁺ adds a partially overlapping atom as a `SELECT DISTINCT` semijoin while the variant drops it (Section 4.2). The Q' -view is completed by joining the P' -views of the bag's children on the bridge variables, and the generator joins only on variables the view already contains, so it never references a column that is not there. For a non-root bag it also emits the P' -view.

Finally the generator emits the `RESULT` view, the reconstruction join of all the Q' -views for a full-result query, or the single `COUNT(*)` scalar for an aggregate. For $Q_\diamond$ the generator produces the views of Section 4.2, up to the tool's own bag numbering.

The decomposer exposes two choices, each of which regenerates the views: switching between IVM⁺ and the variant, and re-rooting the tree at a different bag. Re-rooting changes the parent-child structure of the bags, and so the views and the size of the intermediate results they hold, even though the final result is unchanged. How the orientation affects performance is examined in Chapter 5, where we compare several orientations of the same query.

Figure 4.3 shows the decomposer's web interface on $Q_\diamond$. The schema and the query are entered at the top, on the left and right, and pressing *Decompose* fills in the lower half. On the left, the tree decomposition is drawn with each bag's number and variables and each edge's bridge variables, above a menu that selects the root. On the right are the generated views, with a toggle between IVM⁺ and the variant. The interface labels the variant IVM+* (strict), and the figure shows it, rooted at $\{A, B, D\}$ as in our example. The tool assigns its own bag numbers, so the child bag b_2 appears here as `BAG_1`.

These views are the artifact the experiment harness deploys.



4.3.5 Supported queries and limitations

The decomposer handles *full conjunctive queries* over an input schema, the class the IVM⁺ approach targets. The schema is given as `CREATE TABLE` statements, of which the parser reads only the column definitions and ignores the rest, so the Feldera-specific connector and materialisation annotations of Section 4.2 may be left in place and the same schema used for a pipeline can be handed to the decomposer unchanged. The query, similarly, may be wrapped in a `CREATE [MATERIALIZED] VIEW ...AS` statement, which the parser ignores, so a Feldera view definition can be handed in unchanged too.

A query must be a single `SELECT *` or `SELECT COUNT(*)`. In both cases the decomposer emits the bag views and a final `RESULT` view. For `SELECT *` the `RESULT` view reconstructs the full join, and for `SELECT COUNT(*)` it returns the count, computed on the decomposition by the semiring aggregate of Section 4.2. Tables and columns may be aliased, and a relation may occur several times, so self-joins such as $Q_{\diamond}$ are supported, written either `EDGES AS R1` or with a renamed column list `EDGES AS R1(A, B)`. Joins must be equi-joins, written `JOIN ...ON` or `JOIN ...USING`. The `WHERE` clause may carry filters that each reference a single relation, combined with `AND`, and each such filter is pushed into that relation’s subquery.

Everything outside this class is rejected or unsupported. `COUNT(*)` is the only aggregate, and other aggregates and projections of specific columns are not available. Only equi-joins are recognised, so inequality and outer joins are not supported. A `WHERE` condition that spans more than one relation is rejected, whether a comparison between columns of different relations or a disjunction across them. A filter on each relation separately, joined by `AND`, is the supported form. Subqueries are not supported.

The construction also assumes *set-valued* inputs: on relations with duplicate rows the result can over-count, since an atom lying in several bags is then counted once per bag (Section 4.2).

Finally, the decomposer performs only the checks its translation needs and is not a full SQL validator. A query should therefore be validated in a database engine, for instance Feldera itself, before it is given to the decomposer.

None of these limitations is fundamental: they reflect the tool’s scope as a minimal prototype, and nothing in the approach prevents a future version from supporting the full query surface, further aggregates, and richer joins and filters.

4.4 Experiment Harness

The *experiment harness* is the second tool: it measures how the pipelines of Section 4.2 perform on Feldera. Given a pipeline and a dataset, it deploys the pipeline on Feldera (Section 4.1) and starts it, which ingests the data through the input connectors, and while the pipeline runs it polls Feldera to measure its throughput and peak memory use. It repeats this over many pipeline–dataset combinations, so the experiments of Chapter 5 run unattended and under identical conditions.

Feldera runs inside a Docker container, and the harness is two Python scripts and a small aggregator that drive it, shown in Figure 4.4: an *orchestrator* that steps through the experiments, an *execution* script that runs a single one and measures it, and a *result aggregation* step that collects the measurements. We describe the three in turn. The datasets, the machine the experiments were run on, and the pipeline variants that are compared belong to the evaluation and are introduced in Chapter 5.

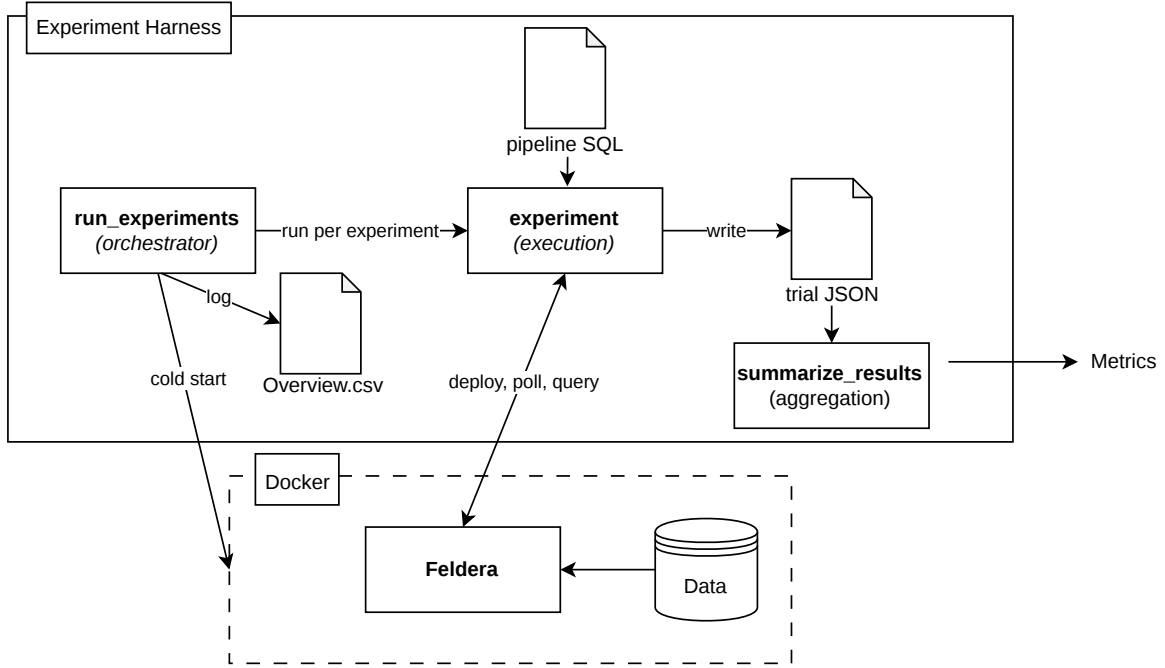


Figure 4.4: Overview of the experiment harness.

4.4.1 Orchestrator

The orchestrator, `run_experiments.py`, holds the list of experiments to run and steps through them one at a time. Each entry names a pipeline (the SQL tables and views of Section 4.2), a dataset, the number of trials, and an output directory for the results.

Before each one it performs a *cold start*, so that every experiment begins from an identical clean state. Restarting the Feldera container clears its in-memory state, but Feldera also keeps its data on disk (Section 4.1), in a directory mounted from the host that outlives the container. The orchestrator therefore also deletes this directory before bringing up a fresh container, so that no data or cache from the previous experiment carries over and distorts the measurement.

The orchestrator then runs the execution script on the experiment under a fifteen-hour timeout, and logs the outcome (completed, timed out, or crashed) and the duration to the overview file `OVERVIEW.csv`, which gives an at-a-glance record of how a long suite of experiments is progressing.

4.4.2 Execution

The execution script, `experiment.py`, runs a single experiment: one pipeline on one dataset, repeated for the requested number of trials. It reads the pipeline’s SQL and substitutes the placeholder `{{FILE-PATH}}` with the dataset path, so the connectors of Section 4.2 read that dataset. One pipeline file can therefore serve several datasets of the same schema. It then deploys the pipeline on Feldera through the Feldera Python SDK and starts it, at which point Feldera ingests the dataset through the input connectors.

Measuring a run means doing two things at once: sampling Feldera’s statistics while waiting for the ingestion to finish. The script therefore uses two threads. A poller samples Feldera every five seconds, recording the pipeline’s uptime, the number of records it has processed, and its resident memory, the amount of RAM it is using. A second thread waits for the ingestion to complete, under a three-hour timeout after which the run is recorded as timed out. Running the two in parallel keeps the polling going for the whole run without blocking on the wait.

Once ingestion finishes, the script queries the `RESULT` view for its row count, or the aggregate value for the aggregate pipelines, as a correctness check: every variant of a query must return the same result, so a disagreeing count exposes a faulty pipeline. From the polled samples it then computes the two performance metrics. The *throughput*, the number of processed records divided by the uptime, measures how fast the pipeline ingests the data and incrementally maintains its views. The *peak memory* is the largest resident memory seen. These, with the result count and the full sample

timeline, are written to a per-trial JSON file in the output directory, and the script clears Feldera's storage between trials, so each trial also starts from a clean state without a full container restart. Because the engine and the data are fixed and the pipelines give the same result, these measurements isolate the effect of rewriting the query as a decomposition. Whether maintaining the query as the decomposition's views is cheaper than as a single flat view is the subject of Chapter 5.

4.4.3 Result aggregation

Each trial leaves one JSON file in the experiment's output directory. A small script, `summarize-results.py`, reads them and reduces a pipeline's trials to a single summary: the average throughput and running time, the peak memory, and the result count. It also flags any pipeline whose trials disagree on the result count or the outcome, which would signal a non-deterministic or faulty run. These summaries are the figures reported in Chapter 5.

Chapter 5

Evaluation

This chapter evaluates the decomposition of Chapter 3, realised as the SQL pipelines of Chapter 4, against a flat-join baseline. The baseline is the flat join query, which Feldera maintains incrementally just as it does the decomposition’s views. The engine and the data are held fixed, so what we isolate is the effect of the rewrite alone: whether maintaining the query as its decomposition’s views is cheaper, in throughput and peak memory, than maintaining it as one flat view. The variants that materialise the full output all return the same result, which doubles as a correctness check (Section 5.2).

The reported metrics and the pipeline notation are defined in full here, expanding the brief description given with the harness (Section 4.4). The experimental setup, datasets, hypotheses, and results then follow in turn.

5.1 Experimental Setup

All experiments were run on `dast-cpu05`, a CPU-only node of the DaST research cluster at the Department of Informatics (IfI), University of Zurich.

The node has two Intel Xeon Silver 4214 processors at 2.20 GHz (12 cores each, for 24 physical cores and 48 hardware threads), 196 GB of main memory and a local SSD. Datasets reside on the node’s local scratch storage (492 GB) and are read locally, so that data access introduces no network overhead. The host runs Debian GNU/Linux 12 (bookworm) with Linux kernel 6.1.0-45-amd64.

The Feldera engine was deployed on the same host from the official `pipeline-manager` Docker image in its default configuration, with no changes to the engine itself other than the volume mounts described in Section 4.4: the container’s storage directory is bind-mounted to host scratch to hold the large intermediate views, and the dataset directory is mounted read-only into the container. We pin Feldera to version 0.286.0 in both `docker-compose.yaml` (the engine image) and `requirements.txt` (the Python client), so that the engine and the harness driving it move together. The host ran Docker Engine 29.7.0 and Python 3.11. The harness runs directly on the host and drives the containerised engine through its REST API. Every variant begins from a cold start, with the engine’s storage wiped between variants (Section 4.4), so that no residual state or cache from an earlier run can influence a measurement.

The datasets were streamed from local CSV files into Feldera through the `file_input` connectors described in Section 4.2, simulating a high-velocity, insert-only ingestion.

5.2 Metrics and Pipelines

5.2.1 Metrics

As described in Section 4.4, the harness samples the running pipeline every five seconds, and once more when it finishes, recording at each sample its uptime, the number of input records processed so far, and its resident-set size (RSS). Every figure we report is derived from this polling. From a finished run we take three metrics: the *time*, the wall-clock duration until the pipeline has ingested the whole dataset and its views are up to date, the *throughput*, the number of input records processed per second, computed as the total records over that uptime, and the *peak memory*, the largest RSS seen across all samples of the run. Alongside these we report the size of the materialised `RESULT` view,

in rows, and the *speedup*, the ratio of the baseline’s time to the variant’s time, which expresses how many times faster (or, when below one, slower) a variant maintains the query than the flat baseline.

Every experiment is run as three independent trials, with the engine’s storage cleared between them so that each trial starts from a clean state. We report the time and throughput as the mean over the three trials, and the peak memory as the maximum over them, so that the memory figure reflects the worst case a run reached rather than a smoothed average. An entry marked **T0** denotes an experiment in which all three trials hit the three-hour timeout without finishing. For such an experiment we still report the peak memory the engine reached and, in parentheses, the mean fraction of the input processed when the timeout struck, but not its output size, throughput, or speedup, since the run never completed. A run that instead crashes after exhausting the storage volume is marked **OOS** and reported the same way. The baseline is the reference for the speedup and so is always $1.00\times$. When the baseline itself times out, its true time exceeds the three-hour timeout, so a variant that finished is reported with a lower-bound speedup $> N\times$, where N is the timeout divided by the variant’s time. Table 5.1 summarises the metrics and how each is aggregated.

Metric	Per trial	Reported
Time	final uptime	mean over the 3 trials
Throughput	records processed $\div$ final uptime	mean over the 3 trials
Peak memory	largest RSS among the trial’s samples	maximum over the 3 trials
Output size	rows in the RESULT view	identical across trials
Speedup	—	baseline time $\div$ variant time

Table 5.1: The reported metrics and how each is aggregated over the three trials. On a run that did not finish, whether timed out (**T0**) or out of storage (**OOS**), only peak memory is reported.

A few measurement artefacts should be kept in mind when reading the tables:

- **Averaging noise.** The reported times are the mean of three trials, so differences on the order of 0.05 min lie within trial-to-trial variance and should not be over-interpreted.
- **Memory sampling.** Peak memory is sampled every five seconds, so for a very short run (for example 0.10 min) the reported value may reflect a single, possibly unrepresentative, snapshot rather than the true peak.

5.2.2 Pipelines and Variants

Every query is run as a flat baseline and as a family of decomposition variants. In the result tables (Tables B.1–B.5) each decomposition variant is named by a compact symbol recording three choices: how the decomposition tree is rooted, whether it uses the standard IVM^+ decomposition or our variant, and how much of the result the pipeline reconstructs.

- **Baseline:** the query as a single flat join, with no decomposition. This is the reference against which the decomposition variants are compared.
- **Orientation** (D_1, D_2): the bag relations and their P' -semijoin filters are defined over a *rooted* tree decomposition (Section 3.1), but the decomposition itself does not fix the root. Re-rooting changes which bag is a parent of which, and so the direction in which the P' -relations filter and the order in which the bags are maintained. D_1 and D_2 denote two orientations of the same decomposition: D_1 is the less balanced one, a deeper, more chain-like tree with the larger relations in the lower bags, and D_2 is the more balanced one, with the bags spread across independent branches and the larger relations nearer the root. Where a decomposition admits only one meaningful orientation, for example a two-bag one, we report only D_1 .
- **IVM^+ (D) or the variant (D'):** the unprimed D builds each bag query the standard IVM^+ way, keeping every atom that shares a variable with the bag and projecting the partially overlapping ones onto the bag’s variables as semijoin filters (Section 3.2). The primed D' is our variant, which drops those projected atoms and keeps only the atoms fully contained in the bag (Section 3.3). The two give the same result and differ only in intermediate size and join work.

- **Reconstruction** (D^{full} , D , D^{count}): D^{full} performs the reconstruction join over all the bags to materialise the complete result (Section 3.1). The plain D , shown indented beneath its D^{full} , stops at the root bag and skips the reconstruction, maintaining only the bag hierarchy. D^{count} returns the size of the full result through the semiring extension, without ever materialising the output (Section 3.4).

After every trial the harness verifies the run by reading the row count of the maintained output view (Section 4.4). For the baseline and the full-reconstruction variants this is the size of the complete result, which they must all agree on. The without-reconstruction variants do not perform the reconstruction, so their terminal view is the root bag and their reported count is that bag’s size rather than the full result, and the count variant is checked against the full result size, which it must reproduce through the semiring extension without materialising the output.

Because the baseline maintains the full join result as a single materialised view, a like-for-like comparison must ask the decomposition to produce that same output. The full-reconstruction variant (D^{full}) does so, re-joining the bags into the complete result, so it and the baseline maintain the same answer and differ only in how they compute it. The without-reconstruction variant (D) instead stops at the root bag and never forms the full output. This is closer to how incremental maintenance is normally used, keeping a compact structure the result can be read from on demand rather than the result itself (Chapter 1), so it is not a like-for-like competitor to the baseline but a measure of how cheaply the decomposition alone can be maintained. The count variant (D^{count}) lies between the two, reporting the result’s size without materialising it.

All experiments were run as pipelines in which every table and every view is materialised, so that any of them can be queried directly. In practice this is often unnecessary, so it is a conservative choice, and one that weighs more heavily on the decomposition, which introduces many additional views. As described in Section 4.1, materialising a table or view can slow a pipeline down, so any advantage the decomposition shows here is obtained despite this. To gauge the effect, we ran a subset of the experiments in which only each pipeline’s terminal view is materialised (Section 4.2). Tables with a primary key stay materialised regardless, since Feldera maintains those automatically (Section 4.1). These runs are reported separately, one table per dataset, in Appendix B.2.

5.3 Datasets and Queries

We evaluate on three classes of dataset, introduced here together with the queries run on each.

5.3.1 Graph datasets

The graph datasets are three directed graphs from the SNAP collection [12], each stored as a single `EDGES` relation. `soc-Epinions1` is a who-trusts-whom social network ($|V| = 75'879$, $|E| = 508'837$), `cit-HepTh` an arXiv high-energy-physics citation network ($|V| = 27'770$, $|E| = 352'807$), and `p2p-Gnutella31` a Gnutella peer-to-peer snapshot ($|V| = 62'586$, $|E| = 147'892$). Every query is a self-join over the edge relation: one acyclic query and several cyclic queries (Table 5.2).

Because every atom is the same `EDGES` relation, the projections that separate IVM^+ from the variant only filter a bag down to the nodes with an edge on one endpoint, almost every node in a dense graph, and so barely filter while still costing extra work. We therefore run only the variant on these datasets (D').

Each query is run under two input configurations. In the first, the edge file is ingested through a single connector into one `EDGES` table, which the query aliases once per atom. In the second, each atom has its own table, all loaded with the same edge file, so the query remains a self-join but over separate copies of the data. Both configurations produce the same result and are reported separately, labelled by the number of connectors. We compare them on time rather than throughput, since the multi-connector runs re-ingest the same file and inflate the record count.

Query	Structure	$w(Q)$
5 Path Query	path, five edges	1
Cyclic Query 1	6-cycle with three chords	$\frac{3}{2}$
Cyclic Query 2	6-cycle with two chords	2
Cyclic Query 3	diamond ($Q_\diamond$)	$\frac{3}{2}$
Cyclic Query 4	6-cycle	2
Cyclic Query 5	5-cycle	2
Cyclic Query 6	4-cycle	2

Table 5.2: The queries run on the graph datasets, with their fractional hypertree width $w(Q)$. Full definitions are given in Appendix A. Cyclic Query 3 is the running example $Q_\diamond$ (Section 2.1).

5.3.2 TPC-H

TPC-H [15] is a standard decision-support benchmark over an eight-relation schema. We generate it at three scale factors, 1, 10, and 30, and run three acyclic queries and one cyclic query (Table 5.3).

Query	Structure	$w(Q)$
3 Table Path Query	path over three tables	1
4 Table Path Query	path over four tables	1
5 Table Path Query	path over five tables	1
Cyclic Query 7	4-cycle over four tables	2

Table 5.3: The queries run on TPC-H, with their fractional hypertree width $w(Q)$. Full definitions are given in Appendix A.

5.3.3 IMDB (JOB)

The Join Order Benchmark (JOB) [11] is derived from the real Internet Movie Database, and unlike the synthetic TPC-H data it contains real-world, correlated data. We run two acyclic queries and one cyclic query (Table 5.4).

Query	Structure	$w(Q)$
Acyclic Query 1	path over five tables	1
Acyclic Query 2	tree over ten tables	1
Cyclic Query 8	3-cycle over three tables	$\frac{3}{2}$

Table 5.4: The queries run on IMDB (JOB), with their fractional hypertree width $w(Q)$. Full definitions are given in Appendix A.

5.4 Hypotheses

Recall the question: is a query cheaper to maintain as its decomposition’s views than as one flat baseline view? The $O(N^{w(Q)})$ bound of Section 2.4 assumes each bag is evaluated with a worst-case-optimal join. Feldera does not implement worst-case-optimal joins [5], so we do not claim to obtain that bound, and whether the decomposition nonetheless outperforms the flat baseline is what the following hypotheses test. The first two concern query structure (H1, H2). The next four concern how cheaply the decomposition itself runs: its reconstruction and without-reconstruction, IVM⁺ and variant, semiring extension, and orientation choices (H3–H6). Two setup choices, the input configuration (Section 5.3) and how much of each pipeline is materialised (Section 5.2.2), are controls on the fairness of the comparison rather than claims about the approach, and we return to them with the results.

H1 (Acyclic queries). As established in Section 2.4, an acyclic query has $w(Q) = 1$: a minimum fractional hypertree width decomposition can give each atom its own bag, so no bag joins more

than one full relation. As shown in Chapter 3, that relation is still combined with the P' -views of the bag’s children (Section 4.2) and, under IVM^+ , with the projected atoms the bag carries (Section 3.2). These only filter tuples from the relation, making it smaller, rather than adding another full relation to the join.

The reconstruction therefore performs the same join as the baseline, but over relations the filters have already reduced, and we accordingly expect the decomposition to be faster than the flat baseline on acyclic queries. Maintaining these views is itself an overhead, though. The decomposition turns each bag into a Q' -view and each non-root bag into a further P' -view, and each keeps the state needed to maintain it even when not materialised (Section 4.1), so we also expect the decomposition to reach a higher peak memory than the baseline, growing with the number of bags. How much the reconstruction itself costs (H3), how IVM^+ compares with its variant (H4), and the semiring extension (H5) are examined separately.

H2 (Cyclic queries). A cyclic query has $w(Q) > 1$, so a minimum fractional hypertree width decomposition must place several atoms in at least one bag (Section 2.4). Such a bag joins those atoms into a closed sub-pattern and passes up a P' -view carrying a joint constraint over several of them, rather than the single-atom projection an acyclic bag would pass. The bags’ Q' -views then enter the reconstruction already filtered and already closed, unlike the baseline, which joins the raw relations directly and may build large intermediates unless Feldera’s optimiser finds a comparably good plan on its own.

We therefore expect the decomposition to run faster than the baseline on cyclic queries, an advantage that comes from these joint-constraint filters and pre-closed sub-patterns, and that should widen with data density and query size.

As in H1, the decomposition maintains a Q' -view per bag and a P' -view per non-root bag, each keeping the state needed to maintain it even when not materialised (Section 4.1), so we again expect a higher peak memory, growing with the number of bags and the size of the multi-relation joins they hold.

H3 (Reconstruction). Every variant has a terminal view. The full-reconstruction variant (D^{full}) ends in the **RESULT** view, the reconstruction join of all the bags’ Q' -views into the complete output (Section 4.2). The without-reconstruction variant (D) instead takes the root bag’s Q' -view as its terminal, so it maintains only the bag hierarchy and never forms the full result. It is therefore not a like-for-like alternative to the baseline (Section 5.2.2), but comparing it with D^{full} isolates the cost of the reconstruction itself.

The two maintain the same bag views and differ only in the reconstruction. That step is itself an incrementally maintained join, updated on every insertion. It joins all the bags’ Q' -views and holds the full output it produces, whereas D stops at the root bag. Because every update that reaches a bag must be propagated through this extra join, and because the full output it stores can be far larger than any single bag, we expect the reconstruction to be a substantial part of the decomposition’s cost in both time and peak memory. The without-reconstruction variant should accordingly be cheaper than the full one, by a margin that grows with the size of the output it avoids. As this concerns the reconstruction step rather than the query’s structure, we expect it to hold for both acyclic and cyclic queries.

H4 (IVM^+ versus the variant). IVM^+ (D) and the variant (D') differ only in each bag’s underlying Q -relation (Section 3.3): IVM^+ keeps the partially overlapping atoms as projected semijoin filters, whereas the variant keeps only the atoms fully contained in the bag. The Q' - and P' -views are built the same way on top of it.

A projected atom either removes tuples the child P' -views would not, or merely repeats filtering they already do. Where it genuinely filters, IVM^+ ’s bag Q' -views hold fewer tuples than the variant’s, so we expect IVM^+ to reach a lower peak memory and, with smaller Q' -views to join, a faster reconstruction. Where the projection is redundant it only adds upkeep, so there we expect the variant to be cheaper. As the two differ in this one definition alone, we expect a modest gap either way, and which side wins is the empirical question Chapter 3 leaves open. We evaluate this comparison on the relational datasets only, since on the graph datasets we maintain just the variant, for the reason given with those datasets (Section 5.3).

H5 (Semiring extension). The count variant (D^{count}) reports the size of the full result through the semiring extension (Section 3.4), without ever materialising it. It maintains the same bag

hierarchy as the without-reconstruction variant (D), with each P' -view now carrying a count beside its bridge variables (Section 4.2).

Like D , it never performs the reconstruction join over all the bags' Q' -views, which H3 expects to be the heavy part of the pipeline. On top of the bag hierarchy it adds only the annotation carried up the tree and the final scalar aggregation. We therefore expect D^{count} to run nearly as cheaply as D and far below the full reconstruction D^{full} , at a peak memory comparable to D , perhaps slightly higher from the annotations it maintains. It does this while still returning a summary of the full result that D , stopping at the root bag, does not. Because this again concerns the reconstruction rather than the query's structure, we expect it to hold for both acyclic and cyclic queries.

H6 (Orientation). A tree decomposition fixes the bags but not the root (Section 3.1). Where a decomposition admits more than one orientation, we evaluate two per query: D_1 , a deep, chain-like tree with the larger relations in the lower bags, and D_2 , a balanced one with them nearer the root (Section 5.2.2). A two-bag decomposition admits only one meaningful orientation, reported as D_1 .

Since a bag's Q' -view is filtered by its children's P' -views, a large relation placed near the root is reduced by the smaller relations beneath it before it reaches the reconstruction, while one placed low stays barely filtered. We therefore expect the balanced D_2 , keeping the larger relations near the root, to feed the reconstruction smaller Q' -views than the chain-like D_1 , which rejoins its large, barely filtered relations. Its balanced shape may also let independent branches be maintained in parallel rather than along one long chain. On both counts we expect D_2 to reach a lower peak memory and run faster.

We are least confident in this prediction of the six: it rests on the P' -views being selective, and where they remove few tuples the orientation should barely matter.

5.5 Results and Discussion

We discuss the results hypothesis by hypothesis, drawing on the main experiment tables in Appendix B.1 (Tables B.1–B.5), in which every table and view is materialised. Every variant that materialises the full result agrees on the output count (Section 4.4), which we take as confirming they compute the same result. After the six hypotheses we return to the two setup controls, the input configuration and how much of each pipeline is materialised, the latter drawing on the terminal-only ablation of Appendix B.2 (Tables B.6–B.10).

5.5.1 H1: Acyclic Queries

H1 predicted the decomposition would be faster than the baseline on acyclic queries and would pay for it in peak memory. The throughput prediction is not borne out. On the TPC-H 4 Table Path Query and 5 Table Path Query, whose output is exactly the size of the driving `LINEITEM` relation and where the baseline finishes, neither the full reconstruction D^{full} nor its variant D'^{full} is ever faster than the baseline: both match it at scale factor 1 and fall away as the data grows, to between 0.61 and $0.62\times$ for D_1^{full} and between 0.72 and $0.74\times$ for $D_1'^{\text{full}}$ at scale factor 30, on both paths. The memory prediction, by contrast, holds: both exceed the baseline's peak memory everywhere, D^{full} by up to $2.6\times$ (for example 42.2 against 16.2 GiB on the 4 Table Path Query at scale factor 10) and the variant by somewhat less, and the gap widens with both the scale factor and the number of tables, that is the number of bags.

This follows from the schema topology of these two queries. Each is a chain of foreign-key to primary-key joins, and referential integrity together with the uniqueness of the primary keys means every child tuple matches exactly one parent, one line item with its order, that order with its customer, and so on. These joins only enrich tuples with further columns, without filtering or multiplying them, so the result is exactly the size of `LINEITEM`. The P' -views and projected atoms therefore have nothing to remove, since with no dangling tuples their semijoins filter nothing (Chapter 3). The reconstruction then reproduces the same enrichment as the baseline over relations it has not shrunk, so it does the baseline's work plus the upkeep of the extra bag views. That upkeep is what the memory figures show, and because it scales with the data while the baseline stays a single lean join, the slowdown grows with the scale factor. Since the projected atoms filter nothing here, dropping them makes the variant the cheaper of the two, which we take up under H4.

The IMDB queries behave differently. Acyclic Query 1, a five-table path, shows no difference from the baseline ($1.00\times$), but Acyclic Query 2, a ten-table tree, is the one acyclic query where the decomposition is faster, in both orientations and for both IVM^+ and the variant, from $1.28\times$ to $1.52\times$, at essentially the baseline’s peak memory (which orientation does better is taken up under H6). Unlike the synthetic TPC-H chains, the real, correlated JOB data does contain dangling tuples, which the decomposition appears to remove early rather than carrying them through a long flat join, an effect we infer from the speedup since we do not measure intermediate sizes directly.

Finally, the TPC-H 3 Table Path Query and the 5 Path Query on the two dense graphs have enormous outputs, up to tens of trillions of rows, and here both the baseline and the full reconstruction time out, so neither produces the full result and only the variants that avoid materialising it finish. That is a property of the reconstruction rather than of the acyclic structure, and we return to it under H3.

5.5.2 H2: Cyclic Queries

H2 predicted the decomposition would be faster than the baseline on cyclic queries, by a margin widening with data density and query size. On the dense graphs this is borne out. The clearest case is Cyclic Query 2 on **cit-HepTh**, where the baseline takes 80.77 min and the full reconstruction maintains the same 266-million-row result in under two minutes, a $42.96\times$ speedup for D_1^{full} and $52.11\times$ for D_2^{full} . Where the baseline times out altogether, the full reconstruction in either orientation still finishes and beats it, by lower bounds from $> 7\times$ on Cyclic Query 4 to $> 191\times$ on Cyclic Query 1. Our running example, the diamond $Q_\diamond$ (Cyclic Query 3, Section 2.1), is faster on both dense graphs, $5.64\times$ on **soc-Epinions1** and $6.68\times$ on **cit-HepTh**. Where the baseline also finishes, this speed does not cost memory, unlike the acyclic case: on **cit-HepTh** Cyclic Query 2 the decomposition’s peak memory is 10–16 GiB against the baseline’s 16, and on the diamond 3.6 against 7.5 GiB, since here it is avoiding a blow-up rather than duplicating a cheap join.

Two kinds of cyclic query on the graphs do not follow this pattern. The first is the four-cycle (Cyclic Query 6), the simplest of them. The baseline maintains it in well under a minute on both dense graphs (0.19 min on **cit-HepTh**), and the full reconstruction is then slower rather than faster, $0.51\times$ on **soc-Epinions1** and $0.70\times$ on **cit-HepTh**, since there is no blow-up for the decomposition to avoid and only its extra views to maintain. The second is the sparse graph **p2p-Gnutella31**, which has so few cycles that its queries return at most sixteen thousand rows, and as few as none. The baseline handles every one in well under a minute, so with no blow-up to avoid the decomposition cannot win: it merely matches the baseline on the near-empty queries and falls behind where its bags still materialise many partial matches that never close into the small result, as on Cyclic Query 4 ($0.31\times$, at 10.7 against 2.4 GiB).

On the denser **soc-Epinions1**, the same cyclic queries have far larger outputs, up to 234 billion rows for Cyclic Query 2, and here even the full reconstruction times out alongside the baseline, since materialising a result of that size is infeasible. The flat plan’s blow-up is still evident, the baseline timing out at 0.21% of the input processed, and the variants that maintain the bag hierarchy without reforming the full result do finish where it and the reconstruction cannot. How cheaply that bag hierarchy alone runs, and why the reconstruction is the step that fails here, is taken up under H3.

The relational datasets reverse the graph picture. On TPC-H Cyclic Query 7, a four-cycle over four tables, the baseline maintains the query comfortably at every scale, from 1.02 to 30.29 min, whereas the decomposition is slower at scale factor 1, $0.71\times$ for D_1^{full} and $0.08\times$ for the variant D_1^{full} , and by scale factor 10 these full reconstructions no longer finish at all, running out of storage or timing out. This is consistent with the mechanism behind the graph wins, now working against the decomposition rather than for it. On these key-based tables the flat join never blows up, its foreign-key to primary-key joins keeping the intermediates small, so the decomposition’s bags have no blow-up to bound and only add the state their own two-table joins must maintain, which Feldera keeps whether or not the views are materialised (Section 4.1). The overhead is thus structural, not a matter of materialising the extra views. The variant fares far worse than IVM^+ , $0.08\times$ against $0.71\times$, because it drops the semijoins that would shrink those bags, a gap we take up under H4.

IMDB Cyclic Query 8, a three-cycle over three tables of real data, is less clear-cut and hinges on the orientation. Rooted one way (D_1) the full reconstruction times out at 63% of the input, rooted the other (D_2) it finishes, and in its best form D_2^{full} it is modestly faster than the baseline, at $1.34\times$. That the decomposition is competitive here at all, unlike on TPC-H, fits the correlated IMDB data giving its filters something to remove, as on the acyclic queries. The orientation dependence itself is

taken up under H6.

Across the datasets, then, H2 holds in a sharpened form. The decomposition is faster than the baseline exactly where the flat plan blows up, on the dense graphs with genuinely large cyclic output, and the advantage grows with density and cycle size, reaching $42.96\times$ on **cit-HepTh** Cyclic Query 2. Where it wins it does so without the memory penalty of the acyclic case, its peak memory comparable to or below the baseline’s, because the blow-up it avoids weighs more heavily on memory than the extra bag views it maintains. Every boundary confirms the same mechanism from the other side: on the sparse graph, the easy four-cycle (Cyclic Query 6), and the key-based TPC-H tables the baseline does not blow up, so the decomposition’s bags have nothing to bound and only add their own state, and there the memory turns into an overhead again. The benefit is therefore specific to cyclic queries over dense data, not to cyclic structure alone. We read this from the run times and peak memory, in particular the baselines that time out early, rather than from intermediate sizes, which we do not measure directly.

5.5.3 H3: Reconstruction

H3 predicted that the reconstruction is a substantial part of the decomposition’s cost, so the without-reconstruction variant should be markedly cheaper than the full reconstruction, by a margin growing with the output it avoids. The results confirm this wherever both finish. On **cit-HepTh** Cyclic Query 2, dropping the reconstruction cuts the time from 1.88 to 0.44 min and the peak memory from 16.1 to 6.8 GiB (D_1^{full} against D_1'), as it stops at the 1.1-million-row root bag rather than reforming the 266-million-row result. The TPC-H 5 Table Path Query makes the point more sharply still. At scale factor 30 its root bag already equals the full output, both 180 million rows, since the reconstruction only enriches each line item with further columns rather than adding tuples (H1). Yet the without-reconstruction D_1 is still faster and lighter than the full reconstruction D_1^{full} , 33.26 against 62.56 min and 29.2 against 51.2 GiB. The cost is thus the reconstruction join and the **RESULT** view it maintains, not merely the size of the result.

In the most pronounced cases the full reconstruction no longer completes at all, while the without-reconstruction variant still finishes. On the TPC-H 3 Table Path Query, whose full result reaches 36 billion rows at scale factor 1, D_1^{full} times out at every scale, whereas D_1 , stopping at a root bag of only ten thousand rows, finishes in 1.02 min. The same pattern appears wherever the reconstructed output reaches into the millions or billions: on **soc-Epinions1** Cyclic Query 2 the full result is 234 billion rows and D_1^{full} times out, while D_1' finishes in 1.41 min, and on **IMDB** Cyclic Query 8 D_1^{full} times out at 63% of the input processed where D_1 finishes in 1.11 min. Here the reconstruction is not merely the larger part of the cost but the part that decides whether the query completes at all.

At the other end, where the full result is small or the whole run is brief, the reconstruction is negligible and the two variants coincide. On **IMDB** Acyclic Query 1 the full reconstruction and the without-reconstruction variant take the same 0.18 min, since the entire run lasts only seconds and reforming the 702-thousand-row result costs nothing measurable. The **p2p-Gnutella31** queries whose results are near-empty behave the same way, with D_1^{full} and D_1' within noise of each other. Across all of this the effect is the same for acyclic and cyclic queries alike, as H3 anticipated: it concerns the reconstruction step, not the shape of the query.

5.5.4 H4: IVM^+ versus the Variant

H4 predicted no uniform winner between IVM^+ and the variant, with the variant cheaper where the projected atoms are redundant and IVM^+ cheaper where they filter. The results bear this out, and on most queries it is the variant that comes out ahead. Wherever the projected atoms filter little, the key-based TPC-H paths and the **IMDB** queries, dropping them leaves the variant as fast as IVM^+ or faster, and consistently lighter, because its bag views drop the projected-atom semijoins and so maintain less state. On the TPC-H 5 Table Path Query at scale factor 30 the variant D_1^{full} runs at $0.72\times$ against IVM^+ ’s $0.62\times$ and at 44.6 against 51.2 GiB, and on the 4 Table Path Query at $0.74\times$ against $0.61\times$. On **IMDB** Acyclic Query 2 the variant is again the faster of the two, $1.52\times$ against $1.44\times$ and 14.4 against 15.1 GiB. As Chapter 3 anticipated, the projected atoms are largely redundant here: either they have little to remove, as on the zero-dangler TPC-H paths (H1), or the child P' -views already remove the same tuples. Either way, dropping them sheds their upkeep at no cost.

The exception is TPC-H Cyclic Query 7, the one query where IVM^+ clearly beats the variant. Here the semijoins genuinely filter, and dropping them lets the variant’s bags grow far larger: at scale factor 1 D_1^{full} falls to $0.08\times$ against IVM^+ ’s $0.71\times$, and its peak memory rises to 31.4 against

14.2 GiB, the one place the variant is the heavier of the two (both fail from scale factor 10 on). This is the case H4 set aside, a cyclic query whose projected atoms remove tuples the P' -views do not, so keeping them pays off. It does not generalise to every cyclic query, though: on IMDB Cyclic Query 8 the semijoins do not help, and the variant D_2^{full} ($1.34\times$) is faster than $\text{IVM}^+ D_2^{\text{full}}$ ($0.40\times$). Whether the projected atoms earn their keep is thus a property of the individual query, not of its being cyclic.

H4’s prediction of no uniform winner holds, but the balance tips clearly toward the variant. On every query but TPC-H Cyclic Query 7 the variant is lighter than IVM^+ and no slower, bar a negligible margin on IMDB Acyclic Query 1, so as a default it is the better choice. On that one query IVM^+ ’s semijoins repay themselves, yet the gain is hollow: both variants stay far slower than the baseline there (H2), so IVM^+ ’s edge over the variant does not make the decomposition competitive. The variant’s advantages, by contrast, appear on queries the decomposition actually wins, such as IMDB Acyclic Query 2 and Cyclic Query 8. The semijoins thus rarely earn their keep, and never turn a losing query into a winning one.

5.5.5 H5: Semiring Extension

H5 predicted the count variant would run nearly as cheaply as the without-reconstruction variant and far below the full reconstruction, since it too skips the reconstruction join and only carries a count up the tree. This holds across the queries. On the diamond $Q_\diamond$ (Cyclic Query 3, Section 2.1), the count finishes in 0.52 min on **soc-Epinions1** and 0.10 on **cit-HepTh**, against 0.44 and 0.10 for the without-reconstruction variant and 0.74 and 0.19 for the full reconstruction. The same appears on **cit-HepTh** Cyclic Query 2, 0.52 min against 0.44 and 1.88, and on IMDB Acyclic Query 2, 1.19 against 1.18 and 2.02. Its peak memory sits a little above the without-reconstruction variant’s on most queries, from the counts it carries, and well below the full reconstruction’s.

Where the full reconstruction cannot complete, the count still can, which shows the advantage of the semiring extension: it reports the size of a result too large to materialise. On IMDB Cyclic Query 8, **soc-Epinions1** Cyclic Query 2 and the TPC-H 3 Table Path Query at scale factor 1 the full reconstruction times out while the count finishes in one to two minutes, returning row counts of 1.9 million, 234 billion and 36 billion without ever forming the result.

The count does not always stay this close to the without-reconstruction variant. On the largest cyclic queries, where the bags hold the most tuples to group, the per-bag grouping and counting that replaces the plain **SELECT DISTINCT** (Section 4.2) adds noticeably to the cost. On **cit-HepTh** Cyclic Query 4 the count takes 15.12 min and 25.3 GiB, above the without-reconstruction variant’s 6.03 min and 22.6 GiB but still well below the full reconstruction’s 24.08 min and 36.6 GiB. On **soc-Epinions1** Cyclic Query 5 it takes 41.83 min against the without-reconstruction variant’s 24.00, where the full reconstruction times out altogether. The count is therefore still the cheaper way to obtain the result’s size, in both time and memory, even on the queries where it costs most.

5.5.6 H6: Orientation

H6 predicted the balanced orientation D_2 would outperform the chain-like D_1 where the P' -filters are selective, while flagging this as the least certain prediction. It is not borne out as a directional rule: neither orientation is uniformly better. The effect is sharpest without the reconstruction, where the bag hierarchy is the whole cost: D_2 leads D_1 by about two to one on **soc-Epinions1** Cyclic Query 5 (11.20 against 24.00 min, 16.0 against 26.9 GiB) and on **cit-HepTh** Cyclic Queries 4 and 5 (3.16 against 6.03 and 0.61 against 1.16 min), yet D_1 wins by an even larger margin on **soc-Epinions1** Cyclic Query 2 (1.41 against 5.19 min). With full reconstruction the margins narrow, and the same split persists: D_2 ahead on **cit-HepTh** Cyclic Queries 2, 4 and 5, D_1 on Cyclic Query 1 and IMDB Acyclic Query 2. On two queries D_2 completes where D_1 times out, **soc-Epinions1** Cyclic Query 4 and IMDB Cyclic Query 8, its one consistent advantage.

The mixed outcome likely reflects a trade-off between two effects of re-rooting, though our measurements cannot confirm it. Re-rooting leaves the bags unchanged and alters only the direction in which the P' -filters flow. In the balanced D_2 the constraints from different parts of the query meet near the root over few steps, so where the filters are selective the heaviest bags are pruned early. In the chain D_1 the filters apply one after another, which instead helps when the bag at the base of the chain is already small on the given data, since the whole chain then stays thin. Which effect dominates depends on the data, not the query alone, which is why Cyclic Query 2 changes winner between **soc-Epinions1** and **cit-HepTh**. Because we observe only time and peak memory, and not the sizes of the intermediate bags, this remains a plausible reading rather than a demonstrated one.

The original rationale for H6, that a balanced tree keeps the larger relations near a filtered root, does not carry over to the graph queries, where every atom is the same **EDGES** relation and orientation is a matter of tree shape alone. What the results do establish is that orientation is a real and at times decisive factor, changing runtime by up to nearly a factor of four and, on two queries, deciding whether the query completes at all. Which orientation is preferable depends on the data as much as the query, however, and is not something our measurements let us predict in advance, so it is best treated as a choice to tune empirically rather than a fixed rule.

5.5.7 Controls

The remaining two settings are controls on the fairness of the comparison rather than tests of the approach. The first is the input configuration of the graph queries. Each is run both through a single connector, ingesting the edge file once into one **EDGES** table aliased per atom, and through one connector per atom, each re-ingesting the same file (Section 5.3). The two produce the same result in essentially the same time and at comparable memory. On **cit-HepTh** Cyclic Query 2 the full reconstruction D_1^{full} takes 1.88 min under either configuration, at 16.1 and 16.4 GiB, and on the diamond $Q_\diamond$ (Cyclic Query 3) on **soc-Epinions1** it takes 0.74 min, at 9.7 and 10.8 GiB. Their throughputs differ in proportion to the number of connectors, but only because each connector re-ingests the file and counts another pass, which is why we compare the configurations on time rather than throughput. Using many connectors adds a small ingestion overhead, as on **soc-Epinions1** Cyclic Query 1, where the without-reconstruction D'_1 takes 0.85 against 1.11 min with nine connectors. That the two are otherwise indistinguishable confirms that the cost of these pipelines lies in maintaining the views, not in ingesting the data. A single connector is nonetheless the better choice in practice, since it reads the data once and avoids even this small overhead.

The second control is how much of each pipeline is materialised. Every result so far comes from pipelines in which every table and view is materialised, a conservative choice that weighs on the decomposition and its many additional views. Repeating a subset of the experiments with only each pipeline’s terminal view materialised (Appendix B.2) leaves every finding of Sections 5.5.1 to 5.5.6 in place. The dense-graph wins persist: the full reconstruction D_1^{full} of the diamond $Q_\diamond$ (Cyclic Query 3) runs at $5.88\times$ on **soc-Epinions1** against $5.64\times$ in the main run, and of **cit-HepTh** Cyclic Query 2 at $41.76\times$ against $42.96\times$. The acyclic TPC-H paths still show no throughput gain, the full reconstruction D_1^{full} remaining below the baseline, and the sparse and relational losses still lose, D_1^{full} on **p2p-Gnutella31** Cyclic Query 4 at $0.30\times$ against $0.31\times$.

Peak memory falls, by a few per cent on TPC-H and up to a quarter on some graph queries, but never enough to change a verdict. The full reconstruction D_1^{full} of the TPC-H 4 Table Path Query at scale factor 30 drops from 47.2 to 43.0 GiB, still far above the baseline’s 24.3. This saving is bounded, though: Feldera maintains each bag view’s state whether or not it is materialised (Section 4.1), so removing the materialisation lightens the pipeline, in memory and often in time, without removing the decomposition’s structural cost. TPC-H Cyclic Query 7 at scale factor 1 makes the point sharply, its full reconstruction moving only from $0.71\times$ to $0.72\times$ and from 14.2 to 13.2 GiB, so its overhead is structural rather than an artefact of materialising the views. That every finding survives the ablation confirms the results are not an artefact of materialising every view. If anything, materialising more than the decomposition needs held it to a slightly harsher standard than a leaner deployment would.

Chapter 6

Conclusion

This thesis asked whether the IVM^+ approach of Abo Khamis et al. is beneficial in practice when realised at the SQL level on an existing incremental engine. We built a query decomposer that rewrites a join query into the bag views of a tree decomposition, in both IVM^+ and an adjusted variant, with a semiring extension for aggregates, and evaluated the rewrites on Feldera against a flat-join baseline across graph, TPC-H and IMDB workloads.

The central finding is that the decomposition helps exactly where the flat query blows up: on cyclic queries over dense data. On **cit-HepTh** the baseline maintains Cyclic Query 2 in 80.77 min, while the decomposition maintains the same 266-million-row result in under two minutes, a 52.11-fold speedup, and at lower peak memory than the baseline, 10.3 against 16 GiB. Our running example, the diamond $Q_\diamond$ (Cyclic Query 3), has its full reconstruction maintained 5.6 to 6.7 times faster on the two dense graphs, or 9.5 to 12.7 times faster without the reconstruction. On **cit-HepTh** Cyclic Queries 1, 4 and 5 the baseline does not finish within three hours at all, whereas the decomposition does. Notably, these gains hold without Feldera implementing the worst-case-optimal joins that the $O(N^{w(Q)})$ bound of Section 2.4 assumes: the advantage comes from the bag structure constraining the query, not from that bound.

The benefit is not universal. Where the flat query does not build large intermediates, the decomposition only adds overhead. On the acyclic TPC-H 4 and 5 Table Path Queries, whose foreign-key to primary-key joins leave nothing for the bags to filter, it matches the baseline’s throughput at best and uses up to 2.6 times its memory. On the sparse **p2p-Gnutella31** graph and on TPC-H Cyclic Query 7, where the baseline stays small, the decomposition is likewise slower. The advantage is specific to cyclic queries over dense data, not to decomposition in general.

Among the variants, the reconstruction is the expensive step. Stopping at the bag hierarchy rather than rejoining it into the full result is markedly cheaper, on **cit-HepTh** Cyclic Query 2 0.44 against 1.88 min, and the semiring extension recovers the result’s size at nearly the same low cost, completing in a fraction of a minute even where the result is far too large for the baseline to materialise, for lower-bound speedups exceeding $1800\times$ and returning counts as large as 234 billion rows at under 13 GiB. Between the two decomposition forms, our variant, which drops the projected-atom semijoins, is the better default, no slower than IVM^+ on every query but TPC-H Cyclic Query 7 and consistently lighter. The tree’s orientation also matters, by up to nearly a factor of four and sometimes deciding whether a query finishes, but which orientation is best depends on the data and is not something our measurements predict.

These results are specific to Feldera and its DBSP engine, and a different engine, in particular one with worst-case-optimal joins, might shift the boundaries. Correctness was checked by comparing the output count across all variants of each query, a strong but not exhaustive guarantee. Future work could extend the semiring aggregation beyond **COUNT** to further aggregates, measure the intermediate bag sizes directly to explain the orientation results our current metrics leave open, and widen the decomposer to the full query surface. Even so, the core question has a clear answer: realised as plain SQL views on an unmodified engine, the IVM^+ decomposition is a worthwhile rewrite for cyclic queries over dense data, the queries where the flat plan would otherwise blow up.

Appendix A

Query Definitions

This appendix gives the full definition of every query used in the evaluation (Chapter 5), written in the join notation of Section 2.1. The corresponding tree decompositions can be reproduced with the query decomposer.

A.1 Graph queries

Every graph query is a self-join over the single edge relation E , so each atom E_i denotes an occurrence of E .

Path Query.

$$E_1(A, B) \bowtie E_2(B, C) \bowtie E_3(C, D) \bowtie E_4(D, E) \bowtie E_5(E, F)$$

Cyclic Query 1.

$$E_1(A, B) \bowtie E_2(B, C) \bowtie E_3(C, D) \bowtie E_4(D, E) \bowtie E_5(E, F) \\ \bowtie E_6(A, F) \bowtie E_7(A, E) \bowtie E_8(A, D) \bowtie E_9(B, D)$$

Cyclic Query 2.

$$E_1(A, B) \bowtie E_2(B, C) \bowtie E_3(C, D) \bowtie E_4(D, E) \bowtie E_5(E, F) \\ \bowtie E_6(A, F) \bowtie E_7(A, E) \bowtie E_8(B, D)$$

Cyclic Query 3 ($Q_\diamond$).

$$E_1(A, B) \bowtie E_2(B, C) \bowtie E_3(C, D) \bowtie E_4(B, D) \bowtie E_5(A, D)$$

Cyclic Query 4.

$$E_1(A, B) \bowtie E_2(B, C) \bowtie E_3(C, D) \bowtie E_4(D, E) \bowtie E_5(E, F) \bowtie E_6(A, F)$$

Cyclic Query 5.

$$E_1(A, B) \bowtie E_2(B, C) \bowtie E_3(C, D) \bowtie E_4(D, E) \bowtie E_5(A, E)$$

Cyclic Query 6.

$$E_1(A, B) \bowtie E_2(B, D) \bowtie E_3(A, E) \bowtie E_4(D, E)$$

A.2 TPC-H queries

For readability, each relation is written with only its join attributes; in the actual queries the relations also carry their remaining attributes. In the 5-Table Path Query the **NATION** relation is used twice, as **NATION_C** (the customer's nation) and **NATION_S** (the supplier's nation), with the corresponding keys **NATIONKEY_C** and **NATIONKEY_S**.

3-Table Path Query.

$$\text{LINEITEM}(\text{SUPPKEY}) \\ \bowtie \text{SUPPLIER}(\text{SUPPKEY}, \text{NATIONKEY}) \\ \bowtie \text{CUSTOMER}(\text{NATIONKEY})$$

4-Table Path Query.

LINEITEM(ORDERKEY, PARTKEY, SUPPKEY)
 ⋈ ORDERS(ORDERKEY, CUSTKEY)
 ⋈ CUSTOMER(CUSTKEY)
 ⋈ PARTSUPP(PARTKEY, SUPPKEY)

5-Table Path Query.

NATION_C(NATIONKEY_C)
 ⋈ CUSTOMER(NATIONKEY_C, CUSTKEY)
 ⋈ ORDERS(CUSTKEY, ORDERKEY)
 ⋈ LINEITEM(ORDERKEY, SUPPKEY)
 ⋈ SUPPLIER(SUPPKEY, NATIONKEY_S)
 ⋈ NATION_S(NATIONKEY_S)

Cyclic Query 7.

LINEITEM(ORDERKEY, SUPPKEY)
 ⋈ ORDERS(ORDERKEY, CUSTKEY)
 ⋈ CUSTOMER(CUSTKEY, NATIONKEY)
 ⋈ SUPPLIER(SUPPKEY, NATIONKEY)

A.3 IMDB (JOB) queries

As above, each relation is written with only its join attributes.

Acyclic Query 1.

company_type(company_type_id)
 ⋈ movie_companies(company_type_id, movie_id)
 ⋈ title(movie_id)
 ⋈ movie_info_idx(movie_id, info_type_id)
 ⋈ info_type(info_type_id)

Acyclic Query 2.

company_name(company_id)
 ⋈ movie_companies(company_id, movie_id)
 ⋈ title(movie_id)
 ⋈ movie_info(movie_id, info_type_id)
 ⋈ info_type(info_type_id)
 ⋈ cast_info(movie_id, char_id, role_id, person_id)
 ⋈ char_name(char_id)
 ⋈ role_type(role_id)
 ⋈ aka_name(person_id)
 ⋈ name(person_id)

Cyclic Query 8.

cast_info(person_id, movie_id)
 ⋈ person_info(person_id, info_type_id)
 ⋈ movie_info(movie_id, info_type_id)

Appendix B

Results Tables

This appendix collects the full performance tables for every query and dataset of the evaluation (Chapter 5). Section B.1 gives the main experiments, in which every table and view is materialised, and Section B.2 gives the ablation, in which only the terminal view is materialised (Section 5.2.2). The variant symbols are defined in Section 5.2.2, and the reported metrics together with the TO and OOS markers in Section 5.2 and Table 5.1.

B.1 Main experiments

Table B.1: Performance comparison on the `soc-Epinions1` dataset.
($|V| = 75'879$, $|E| = 508'837$)

Variant	Output Size	Time	Throughput	Speedup	Peak RAM
<i>5 Path Query</i>					
Single Input Connector					
Baseline	–	TO (0.21%)	–	–	8.6GiB
D_1^{full}	–	TO (0.21%)	–	–	9.1GiB
D_1'	471'881r	0.10min	81'663r/s	>1800x	0.6GiB
D_1^{count}	39'501'979'009'022r	0.10min	80'935r/s	>1800x	1.4GiB
D_2^{full}	–	TO (0.21%)	–	–	40.2GiB
D_2'	449'932r	0.10min	81'445r/s	>1800x	0.6GiB
D_2^{count}	39'501'979'009'022r	0.10min	81'103r/s	>1800x	1.1GiB
5 Input Connectors					
Baseline	–	TO (0.21%)	–	–	8.7GiB
D_1^{full}	–	TO (0.21%)	–	–	9.8GiB
D_1'	471'881r	0.10min	410'822r/s	>1800x	0.9GiB
D_1^{count}	39'501'979'009'022r	0.10min	412'064r/s	>1800x	1.8GiB
D_2^{full}	–	TO (0.21%)	–	–	39.9GiB
D_2'	449'932r	0.10min	410'670r/s	>1800x	0.9GiB
D_2^{count}	39'501'979'009'022r	0.10min	416'639r/s	>1800x	1.4GiB
<i>Cyclic Query 1</i>					
Single Input Connector					
Baseline	–	TO (0.21%)	–	–	27.9GiB
D_1^{full}	–	TO (0.21%)	–	–	44.9GiB
D_1'	3'505'469r	0.85min	9926r/s	>211x	8.0GiB
D_1^{count}	121'124'217'498r	1.02min	8303r/s	>176x	9.2GiB
D_2^{full}	–	TO (0.21%)	–	–	46.1GiB
D_2'	3'425'880r	1.47min	5787r/s	>122x	8.6GiB
D_2^{count}	121'124'217'498r	3.14min	2706r/s	>57x	9.0GiB

Variant	Output Size	Time	Throughput	Speedup	Peak RAM
9 Input Connectors					
Baseline	–	TO (0.21%)	–	–	22.5GiB
D_1^{full}	–	TO (1.56%)	–	–	47.2GiB
D_1'	3'505'469r	1.11min	69'056r/s	>162x	10.4GiB
D_1^{count}	121'124'217'498r	1.30min	58'803r/s	>138x	11.3GiB
D_2^{full}	–	TO (0.21%)	–	–	46.3GiB
D_2'	3'425'880r	1.61min	47'535r/s	>111x	9.2GiB
D_2^{count}	121'124'217'498r	3.05min	25'026r/s	>59x	9.6GiB
<i>Cyclic Query 2</i>					
Single Input Connector					
Baseline	–	TO (0.21%)	–	–	28.8GiB
D_1^{full}	–	TO (0.21%)	–	–	44.8GiB
D_1'	3'544'194r	1.41min	6019r/s	>127x	10.8GiB
D_1^{count}	234'067'755'356r	1.77min	4786r/s	>101x	12.7GiB
D_2^{full}	–	TO (5.03%)	–	–	87.4GiB
D_2'	28'048'050r	5.19min	1633r/s	>34x	13.5GiB
D_2^{count}	234'067'755'356r	12.93min	656r/s	>13x	17.2GiB
8 Input Connectors					
Baseline	–	TO (0.21%)	–	–	28.9GiB
D_1^{full}	–	TO (0.21%)	–	–	45.3GiB
D_1'	3'544'194r	1.44min	47'149r/s	>125x	11.5GiB
D_1^{count}	234'067'755'356r	1.83min	37'129r/s	>98x	13.7GiB
D_2^{full}	–	TO (2.94%)	–	–	50.1GiB
D_2'	28'048'050r	5.20min	13'057r/s	>34x	13.7GiB
D_2^{count}	234'067'755'356r	12.96min	5236r/s	>13x	18.1GiB
<i>Cyclic Query 3</i>					
Single Input Connector					
Baseline	100'317'892r	4.17min	2036r/s	1.00x	9.5GiB
D_1^{full}	100'317'892r	0.74min	11'418r/s	5.64x	9.7GiB
D_1'	3'548'205r	0.44min	19'334r/s	9.48x	6.2GiB
D_1^{count}	100'317'892r	0.52min	16'310r/s	8.02x	6.6GiB
5 Input Connectors					
Baseline	100'317'892r	4.17min	10'182r/s	1.00x	10.0GiB
D_1^{full}	100'317'892r	0.74min	57'170r/s	5.64x	10.8GiB
D_1'	3'548'205r	0.47min	91'565r/s	8.87x	6.7GiB
D_1^{count}	100'317'892r	0.52min	81'499r/s	8.02x	6.9GiB
<i>Cyclic Query 4</i>					
Single Input Connector					
Baseline	–	TO (0.21%)	–	–	9.5GiB
D_1^{full}	–	TO (0.21%)	–	–	56.6GiB
D_1'	–	TO (78.82%)	–	–	82.3GiB
D_1^{count}	–	TO (31.66%)	–	–	58.5GiB
D_2^{full}	–	TO (0.21%)	–	–	63.7GiB
D_2'	970'903'178r	77.86min	109r/s	>2x	43.9GiB
D_2^{count}	–	TO (31.66%)	–	–	43.7GiB
6 Input Connectors					
Baseline	–	TO (0.21%)	–	–	39.9GiB
D_1^{full}	–	TO (0.21%)	–	–	67.5GiB
D_1'	–	TO (78.82%)	–	–	80.9GiB
D_1^{count}	–	TO (31.66%)	–	–	59.8GiB
D_2^{full}	–	TO (0.21%)	–	–	62.5GiB
D_2'	970'903'178r	77.64min	655r/s	>2x	43.7GiB

Variant	Output Size	Time	Throughput	Speedup	Peak RAM
D_2^{count}	–	TO (31.66%)	–	–	42.8GiB
<i>Cyclic Query 5</i>					
Single Input Connector					
Baseline	–	TO (7.59%)	–	–	18.9GiB
D_1^{full}	–	TO (15.93%)	–	–	63.2GiB
D_1'	36'104'526r	24.00min	353r/s	>7x	26.9GiB
D_1^{count}	25'906'152'819r	41.83min	203r/s	>4x	37.9GiB
D_2^{full}	–	TO (15.93%)	–	–	54.0GiB
D_2'	454'447'219r	11.20min	757r/s	>16x	16.0GiB
D_2^{count}	25'906'152'819r	22.33min	380r/s	>8x	20.2GiB
5 Input Connectors					
Baseline	–	TO (4.35%)	–	–	16.3GiB
D_1^{full}	–	TO (15.93%)	–	–	59.8GiB
D_1'	36'104'526r	24.09min	1761r/s	>7x	26.3GiB
D_1^{count}	25'906'152'819r	41.52min	1021r/s	>4x	37.3GiB
D_2^{full}	–	TO (15.92%)	–	–	53.4GiB
D_2'	454'447'219r	11.37min	3730r/s	>15x	16.0GiB
D_2^{count}	25'906'152'819r	22.61min	1876r/s	>7x	20.4GiB
<i>Cyclic Query 6</i>					
Single Input Connector					
Baseline	305'020'011r	1.47min	5777r/s	1.00x	13.3GiB
D_1^{full}	305'020'011r	2.86min	2966r/s	0.51x	14.7GiB
D_1'	30'482'554r	0.94min	9032r/s	1.56x	8.8GiB
D_1^{count}	305'020'011r	1.36min	6253r/s	1.08x	11.0GiB
4 Input Connectors					
Baseline	305'020'011r	1.58min	21'528r/s	1.00x	15.5GiB
D_1^{full}	305'020'011r	2.75min	12'351r/s	0.57x	15.4GiB
D_1'	30'482'554r	1.02min	33'176r/s	1.55x	9.6GiB
D_1^{count}	305'020'011r	1.36min	25'028r/s	1.16x	11.1GiB

Table B.2: Performance comparison on the **cit-HepTh** dataset.
 $(|V| = 27'770, |E| = 352'807)$

Variant	Output Size	Time	Throughput	Speedup	Peak RAM
<i>5 Path Query</i>					
Single Input Connector					
Baseline	–	TO (27.57%)	–	–	45.4GiB
D_1^{full}	–	TO (7.55%)	–	–	39.2GiB
D_1'	281'389r	0.11min	55'742r/s	>1636x	0.5GiB
D_1^{count}	67'450'796'768r	0.10min	56'548r/s	>1800x	0.9GiB
D_2^{full}	–	TO (27.82%)	–	–	49.1GiB
D_2'	267'650r	0.11min	55'981r/s	>1636x	0.5GiB
D_2^{count}	67'450'796'768r	0.10min	56'565r/s	>1800x	0.7GiB
5 Input Connectors					
Baseline	–	TO (26.16%)	–	–	43.5GiB
D_1^{full}	–	TO (28.25%)	–	–	46.5GiB
D_1'	281'389r	0.10min	284'237r/s	>1800x	0.8GiB
D_1^{count}	67'450'796'768r	0.10min	286'378r/s	>1800x	1.2GiB
D_2^{full}	–	TO (26.02%)	–	–	45.0GiB
D_2'	267'650r	0.10min	284'299r/s	>1800x	0.7GiB
D_2^{count}	67'450'796'768r	0.10min	286'464r/s	>1800x	1.1GiB

Variant	Output Size	Time	Throughput	Speedup	Peak RAM
---------	-------------	------	------------	---------	----------

Cyclic Query 1

Single Input Connector

Baseline	–	TO (22.82%)	–	–	32.5GiB
D_1^{full}	161'417'942r	0.94min	6276r/s	>191x	7.7GiB
D_1'	878'464r	0.35min	16'581r/s	>514x	5.1GiB
D_1^{count}	161'417'942r	0.41min	14'516r/s	>439x	5.3GiB
D_2^{full}	161'417'942r	1.02min	5760r/s	>176x	9.0GiB
D_2'	822'847r	0.44min	13'486r/s	>409x	6.2GiB
D_2^{count}	161'417'942r	0.60min	9723r/s	>300x	6.7GiB

9 Input Connectors

Baseline	–	TO (22.82%)	–	–	31.8GiB
D_1^{full}	161'417'942r	1.11min	47'875r/s	>162x	9.5GiB
D_1'	878'464r	0.58min	92'104r/s	>310x	7.0GiB
D_1^{count}	161'417'942r	0.60min	87'644r/s	>300x	7.3GiB
D_2^{full}	161'417'942r	1.02min	51'850r/s	>176x	9.5GiB
D_2'	822'847r	0.52min	101'663r/s	>346x	6.5GiB
D_2^{count}	161'417'942r	0.60min	87'720r/s	>300x	7.2GiB

Cyclic Query 2

Single Input Connector

Baseline	266'404'327r	80.77min	73r/s	1.00x	16.0GiB
D_1^{full}	266'404'327r	1.88min	3122r/s	42.96x	16.1GiB
D_1'	1'105'455r	0.44min	13'443r/s	183.57x	6.8GiB
D_1^{count}	266'404'327r	0.52min	11'290r/s	155.33x	7.7GiB
D_2^{full}	266'404'327r	1.55min	3796r/s	52.11x	10.3GiB
D_2'	2'023'391r	0.61min	9719r/s	132.41x	7.4GiB
D_2^{count}	266'404'327r	0.77min	7622r/s	104.90x	7.8GiB

8 Input Connectors

Baseline	266'404'327r	79.83min	589r/s	1.00x	16.6GiB
D_1^{full}	266'404'327r	1.88min	24'979r/s	42.46x	16.4GiB
D_1'	1'105'455r	0.46min	101'895r/s	173.54x	7.5GiB
D_1^{count}	266'404'327r	0.52min	90'120r/s	153.52x	8.2GiB
D_2^{full}	266'404'327r	1.52min	30'924r/s	52.52x	11.0GiB
D_2'	2'023'391r	0.60min	77'834r/s	133.05x	7.7GiB
D_2^{count}	266'404'327r	0.77min	61'006r/s	103.68x	8.3GiB

Cyclic Query 3

Single Input Connector

Baseline	10'050'591r	1.27min	4619r/s	1.00x	7.5GiB
D_1^{full}	10'050'591r	0.19min	31'291r/s	6.68x	3.6GiB
D_1'	1'270'167r	0.10min	56'475r/s	12.70x	1.8GiB
D_1^{count}	10'050'591r	0.10min	56'591r/s	12.70x	2.0GiB

5 Input Connectors

Baseline	10'050'591r	1.36min	21'685r/s	1.00x	7.5GiB
D_1^{full}	10'050'591r	0.19min	157'103r/s	7.16x	4.1GiB
D_1'	1'270'167r	0.10min	283'887r/s	13.60x	2.2GiB
D_1^{count}	10'050'591r	0.13min	244'023r/s	10.46x	2.3GiB

Cyclic Query 4

Single Input Connector

Baseline	–	TO (49.12%)	–	–	16.4GiB
D_1^{full}	2'388'153'166r	24.08min	244r/s	>7x	36.6GiB
D_1'	4'314'290r	6.03min	975r/s	>29x	22.6GiB
D_1^{count}	2'388'153'166r	15.12min	389r/s	>11x	25.3GiB
D_2^{full}	2'388'153'166r	19.07min	308r/s	>9x	30.5GiB

Variant	Output Size	Time	Throughput	Speedup	Peak RAM
D_2'	22'049'456r	3.16min	1858r/s	>56x	16.3GiB
D_2^{count}	2'388'153'166r	7.00min	840r/s	>25x	21.5GiB
6 Input Connectors					
Baseline	–	TO (48.01%)	–	–	17.3GiB
D_1^{full}	2'388'153'166r	24.72min	1428r/s	>7x	37.6GiB
D_1'	4'314'290r	6.03min	5851r/s	>29x	22.8GiB
D_1^{count}	2'388'153'166r	14.93min	2364r/s	>12x	27.3GiB
D_2^{full}	2'388'153'166r	19.07min	1851r/s	>9x	29.3GiB
D_2'	22'049'456r	3.22min	10'983r/s	>55x	17.1GiB
D_2^{count}	2'388'153'166r	7.36min	4795r/s	>24x	21.9GiB
<i>Cyclic Query 5</i>					
Single Input Connector					
Baseline	–	TO (49.61%)	–	–	14.2GiB
D_1^{full}	188'370'277r	2.05min	2867r/s	>87x	19.9GiB
D_1'	3'756'194r	1.16min	5070r/s	>155x	12.2GiB
D_1^{count}	188'370'277r	2.02min	2906r/s	>89x	15.5GiB
D_2^{full}	188'370'277r	1.47min	4010r/s	>122x	17.3GiB
D_2'	11'132'518r	0.61min	9714r/s	>295x	8.5GiB
D_2^{count}	188'370'277r	1.02min	5749r/s	>176x	9.5GiB
5 Input Connectors					
Baseline	–	TO (48.03%)	–	–	14.8GiB
D_1^{full}	188'370'277r	2.05min	14'343r/s	>87x	21.2GiB
D_1'	3'756'194r	1.16min	25'379r/s	>155x	12.4GiB
D_1^{count}	188'370'277r	2.05min	14'342r/s	>87x	15.0GiB
D_2^{full}	188'370'277r	1.52min	19'339r/s	>118x	16.3GiB
D_2'	11'132'518r	0.66min	44'702r/s	>272x	9.1GiB
D_2^{count}	188'370'277r	1.05min	28'030r/s	>171x	9.6GiB
<i>Cyclic Query 6</i>					
Single Input Connector					
Baseline	15'629'160r	0.19min	31'214r/s	1.00x	4.4GiB
D_1^{full}	15'629'160r	0.27min	21'609r/s	0.70x	7.2GiB
D_1'	2'809'865r	0.19min	31'193r/s	1.00x	4.0GiB
D_1^{count}	15'629'160r	0.19min	31'092r/s	1.00x	5.2GiB
4 Input Connectors					
Baseline	15'629'160r	0.19min	124'253r/s	1.00x	4.9GiB
D_1^{full}	15'629'160r	0.27min	86'679r/s	0.70x	7.3GiB
D_1'	2'809'865r	0.19min	125'073r/s	1.00x	4.2GiB
D_1^{count}	15'629'160r	0.22min	111'999r/s	0.86x	5.2GiB

Table B.3: Performance comparison on the **p2p-Gnutella31** dataset.
 $(|V| = 62'586, |E| = 147'892)$

Variant	Output Size	Time	Throughput	Speedup	Peak RAM
<i>5 Path Query</i>					
Single Input Connector					
Baseline	26'632'138r	0.19min	13'048r/s	1.00x	6.0GiB
D_1^{full}	26'632'138r	0.19min	13'221r/s	1.00x	6.2GiB
D_1'	143'932r	0.10min	23'816r/s	1.90x	0.4GiB
D_1^{count}	26'632'138r	0.10min	23'627r/s	1.90x	0.6GiB
D_2^{full}	26'632'138r	0.19min	13'154r/s	1.00x	5.6GiB
D_2'	52'046r	0.10min	23'548r/s	1.90x	0.4GiB

Variant	Output Size	Time	Throughput	Speedup	Peak RAM
D_2^{count}	26'632'138r	0.10min	23'861r/s	1.90x	0.5GiB
5 Input Connectors					
Baseline	26'632'138r	0.19min	65'527r/s	1.00x	5.9GiB
D_1^{full}	26'632'138r	0.19min	65'990r/s	1.00x	6.3GiB
D_1'	143'932r	0.10min	117'493r/s	1.90x	0.6GiB
D_1^{count}	26'632'138r	0.10min	119'615r/s	1.90x	0.8GiB
D_2^{full}	26'632'138r	0.16min	82'712r/s	1.19x	6.2GiB
D_2'	52'046r	0.10min	120'589r/s	1.90x	0.6GiB
D_2^{count}	26'632'138r	0.10min	118'422r/s	1.90x	0.7GiB
<i>Cyclic Query 1</i>					
Single Input Connector					
Baseline	0r	0.52min	4722r/s	1.00x	7.5GiB
D_1^{full}	0r	0.10min	23'998r/s	5.20x	0.6GiB
D_1'	0r	0.10min	23'655r/s	5.20x	0.6GiB
D_1^{count}	0r	0.10min	23'993r/s	5.20x	0.6GiB
D_2^{full}	0r	0.10min	23'970r/s	5.20x	0.9GiB
D_2'	0r	0.10min	23'816r/s	5.20x	0.9GiB
D_2^{count}	0r	0.10min	23'665r/s	5.20x	0.9GiB
9 Input Connectors					
Baseline	0r	0.52min	42'660r/s	1.00x	8.3GiB
D_1^{full}	0r	0.10min	212'286r/s	5.20x	1.0GiB
D_1'	0r	0.10min	213'836r/s	5.20x	1.0GiB
D_1^{count}	0r	0.10min	215'329r/s	5.20x	1.0GiB
D_2^{full}	0r	0.10min	215'560r/s	5.20x	1.1GiB
D_2'	0r	0.10min	216'734r/s	5.20x	1.1GiB
D_2^{count}	0r	0.10min	216'602r/s	5.20x	1.1GiB
<i>Cyclic Query 2</i>					
Single Input Connector					
Baseline	1r	0.19min	13'125r/s	1.00x	4.5GiB
D_1^{full}	1r	0.10min	23'898r/s	1.90x	0.6GiB
D_1'	1r	0.10min	24'057r/s	1.90x	0.6GiB
D_1^{count}	1r	0.10min	24'033r/s	1.90x	0.6GiB
D_2^{full}	1r	0.10min	23'927r/s	1.90x	0.5GiB
D_2'	1r	0.10min	23'684r/s	1.90x	0.5GiB
D_2^{count}	1r	0.10min	23'766r/s	1.90x	0.5GiB
8 Input Connectors					
Baseline	1r	0.19min	106'109r/s	1.00x	4.9GiB
D_1^{full}	1r	0.10min	192'401r/s	1.90x	0.9GiB
D_1'	1r	0.10min	189'541r/s	1.90x	1.0GiB
D_1^{count}	1r	0.10min	191'811r/s	1.90x	1.0GiB
D_2^{full}	1r	0.10min	191'228r/s	1.90x	0.9GiB
D_2'	1r	0.10min	189'510r/s	1.90x	0.9GiB
D_2^{count}	1r	0.10min	192'871r/s	1.90x	0.9GiB
<i>Cyclic Query 3</i>					
Single Input Connector					
Baseline	51r	0.11min	23'340r/s	1.00x	0.6GiB
D_1^{full}	51r	0.11min	23'408r/s	1.00x	0.6GiB
D_1'	50r	0.11min	23'366r/s	1.00x	0.6GiB
D_1^{count}	51r	0.10min	23'832r/s	1.10x	0.6GiB
5 Input Connectors					
Baseline	51r	0.10min	120'885r/s	1.00x	0.7GiB
D_1^{full}	51r	0.10min	122'537r/s	1.00x	0.7GiB

Variant	Output Size	Time	Throughput	Speedup	Peak RAM
D'_1	50r	0.10min	118'508r/s	1.00x	0.7GiB
D_1^{count}	51r	0.10min	118'219r/s	1.00x	0.7GiB
<i>Cyclic Query 4</i>					
Single Input Connector					
Baseline	16'058r	0.11min	23'419r/s	1.00x	2.4GiB
D_1^{full}	16'058r	0.35min	6974r/s	0.31x	10.7GiB
D'_1	9672r	0.27min	9058r/s	0.41x	9.1GiB
D_1^{count}	16'058r	0.35min	6952r/s	0.31x	9.3GiB
D_2^{full}	16'058r	0.19min	13'215r/s	0.58x	4.3GiB
D'_2	10'268r	0.19min	13'101r/s	0.58x	3.7GiB
D_2^{count}	16'058r	0.19min	13'134r/s	0.58x	4.5GiB
6 Input Connectors					
Baseline	16'058r	0.11min	140'582r/s	1.00x	2.5GiB
D_1^{full}	16'058r	0.35min	41'859r/s	0.31x	11.0GiB
D'_1	9672r	0.27min	54'575r/s	0.41x	9.4GiB
D_1^{count}	16'058r	0.36min	41'599r/s	0.31x	9.8GiB
D_2^{full}	16'058r	0.19min	79'165r/s	0.58x	4.6GiB
D'_2	10'268r	0.19min	78'580r/s	0.58x	4.0GiB
D_2^{count}	16'058r	0.19min	78'861r/s	0.58x	4.7GiB
<i>Cyclic Query 5</i>					
Single Input Connector					
Baseline	5674r	0.11min	23'409r/s	1.00x	2.7GiB
D_1^{full}	5674r	0.10min	23'553r/s	1.10x	2.8GiB
D'_1	3763r	0.10min	23'687r/s	1.10x	2.4GiB
D_1^{count}	5674r	0.10min	23'986r/s	1.10x	2.9GiB
D_2^{full}	5674r	0.10min	23'896r/s	1.10x	1.5GiB
D'_2	3816r	0.11min	23'380r/s	1.00x	1.3GiB
D_2^{count}	5674r	0.10min	23'594r/s	1.10x	1.5GiB
5 Input Connectors					
Baseline	5674r	0.10min	118'643r/s	1.00x	2.5GiB
D_1^{full}	5674r	0.10min	118'505r/s	1.00x	3.2GiB
D'_1	3763r	0.10min	119'280r/s	1.00x	2.6GiB
D_1^{count}	5674r	0.10min	118'974r/s	1.00x	3.0GiB
D_2^{full}	5674r	0.10min	118'713r/s	1.00x	1.6GiB
D'_2	3816r	0.10min	118'433r/s	1.00x	1.6GiB
D_2^{count}	5674r	0.10min	120'787r/s	1.00x	1.7GiB
<i>Cyclic Query 6</i>					
Single Input Connector					
Baseline	1404r	0.10min	23'498r/s	1.00x	0.5GiB
D_1^{full}	1404r	0.10min	23'740r/s	1.00x	0.9GiB
D'_1	1099r	0.11min	23'376r/s	0.91x	0.9GiB
D_1^{count}	1404r	0.10min	23'806r/s	1.00x	1.0GiB
4 Input Connectors					
Baseline	1404r	0.10min	93'957r/s	1.00x	0.6GiB
D_1^{full}	1404r	0.10min	95'015r/s	1.00x	1.2GiB
D'_1	1099r	0.10min	94'049r/s	1.00x	1.1GiB
D_1^{count}	1404r	0.10min	95'818r/s	1.00x	1.1GiB

Table B.4: Performance comparison on the TPC-H dataset.

Variant	Output Size	Time	Throughput	Speedup	Peak RAM
<i>4 Table Path Query</i>					
Scale Factor 1 (~1 GB)					
Baseline	6'001'215r	1.02min	137'629r/s	1.00x	9.4GiB
D_1^{full}	6'001'215r	1.02min	137'972r/s	1.00x	16.1GiB
D_1	6'001'215r	1.02min	137'896r/s	1.00x	10.9GiB
D_1^{count}	6'001'215r	1.05min	134'709r/s	0.97x	11.6GiB
$D_1^{\text{full}'}\prime$	6'001'215r	1.02min	137'918r/s	1.00x	12.4GiB
D_1'	6'001'215r	0.99min	141'859r/s	1.03x	9.6GiB
$D_1^{\text{count}'}\prime$	6'001'215r	1.02min	138'098r/s	1.00x	10.4GiB
Scale Factor 10 (~10 GB)					
Baseline	59'986'052r	10.34min	136'260r/s	1.00x	16.2GiB
D_1^{full}	59'986'052r	12.29min	114'637r/s	0.84x	42.2GiB
D_1	59'986'052r	10.26min	137'329r/s	1.01x	24.2GiB
D_1^{count}	59'986'052r	10.34min	136'156r/s	1.00x	27.0GiB
$D_1^{\text{full}'}\prime$	59'986'052r	11.87min	118'835r/s	0.87x	32.8GiB
D_1'	59'986'052r	9.95min	141'529r/s	1.04x	12.4GiB
$D_1^{\text{count}'}\prime$	59'986'052r	10.03min	140'374r/s	1.03x	14.3GiB
Scale Factor 30 (~30 GB)					
Baseline	179'998'372r	33.57min	125'855r/s	1.00x	30.2GiB
D_1^{full}	179'998'372r	54.60min	77'386r/s	0.61x	47.2GiB
D_1	179'998'372r	34.48min	122'542r/s	0.97x	32.7GiB
D_1^{count}	179'998'372r	35.07min	120'488r/s	0.96x	33.9GiB
$D_1^{\text{full}'}\prime$	179'998'372r	45.17min	93'575r/s	0.74x	44.5GiB
D_1'	179'998'372r	29.98min	140'936r/s	1.12x	19.3GiB
$D_1^{\text{count}'}\prime$	179'998'372r	30.17min	140'035r/s	1.11x	19.9GiB
<i>5 Table Path Query</i>					
Scale Factor 1 (~1 GB)					
Baseline	6'001'215r	1.02min	124'981r/s	1.00x	9.6GiB
D_1^{full}	6'001'215r	1.05min	121'952r/s	0.97x	16.9GiB
D_1	6'001'215r	1.02min	125'276r/s	1.00x	11.6GiB
D_1^{count}	6'001'215r	1.02min	125'342r/s	1.00x	12.4GiB
$D_1^{\text{full}'}\prime$	6'001'215r	1.02min	124'973r/s	1.00x	14.5GiB
D_1'	6'001'215r	1.02min	125'132r/s	1.00x	9.2GiB
$D_1^{\text{count}'}\prime$	6'001'215r	1.02min	125'127r/s	1.00x	9.7GiB
Scale Factor 10 (~10 GB)					
Baseline	59'986'052r	11.12min	114'865r/s	1.00x	21.0GiB
D_1^{full}	59'986'052r	13.93min	91'718r/s	0.80x	47.8GiB
D_1	59'986'052r	10.06min	126'869r/s	1.11x	25.2GiB
D_1^{count}	59'986'052r	10.37min	123'166r/s	1.07x	27.1GiB
$D_1^{\text{full}'}\prime$	59'986'052r	13.09min	97'589r/s	0.85x	38.0GiB
D_1'	59'986'052r	9.81min	130'111r/s	1.13x	13.0GiB
$D_1^{\text{count}'}\prime$	59'986'052r	10.01min	127'591r/s	1.11x	13.9GiB
Scale Factor 30 (~30 GB)					
Baseline	179'998'372r	38.55min	99'474r/s	1.00x	30.7GiB
D_1^{full}	179'998'372r	62.56min	61'234r/s	0.62x	51.2GiB
D_1	179'998'372r	33.26min	115'175r/s	1.16x	29.2GiB
D_1^{count}	179'998'372r	34.35min	111'527r/s	1.12x	31.5GiB
$D_1^{\text{full}'}\prime$	179'998'372r	53.76min	71'289r/s	0.72x	44.6GiB
D_1'	179'998'372r	29.45min	130'074r/s	1.31x	17.5GiB
$D_1^{\text{count}'}\prime$	179'998'372r	29.90min	128'111r/s	1.29x	19.9GiB
<i>3 Table Path Query</i>					

Variant	Output Size	Time	Throughput	Speedup	Peak RAM
Scale Factor 1 (~1 GB)					
Baseline	–	TO (6.65%)	–	–	23.6GiB
D_1^{full}	–	TO (5.45%)	–	–	23.8GiB
D_1	10'000r	1.02min	100'500r/s	>176x	8.6GiB
D_1^{count}	36'007'310'944r	1.02min	100'584r/s	>176x	8.7GiB
D_1^{full}	–	TO (5.72%)	–	–	23.5GiB
D_1'	10'000r	0.97min	106'446r/s	>185x	7.3GiB
D_1^{count}	36'007'310'944r	0.97min	106'428r/s	>185x	7.3GiB
Scale Factor 10 (~10 GB)					
Baseline	–	TO (0.55%)	–	–	24.9GiB
D_1^{full}	–	TO (0.52%)	–	–	25.0GiB
D_1	100'000r	9.75min	105'227r/s	>18x	10.7GiB
D_1^{count}	3'599'144'795'238r	10.51min	97'748r/s	>17x	12.5GiB
D_1^{full}	–	TO (0.55%)	–	–	26.1GiB
D_1'	100'000r	9.48min	108'326r/s	>18x	13.4GiB
D_1^{count}	3'599'144'795'238r	9.70min	105'820r/s	>18x	9.9GiB
Scale Factor 30 (~30 GB)					
Baseline	–	TO (0.29%)	–	–	26.5GiB
D_1^{full}	–	TO (0.30%)	–	–	26.7GiB
D_1	300'000r	29.70min	103'718r/s	>6x	12.3GiB
D_1^{count}	32'399'833'908'509r	31.81min	96'847r/s	>5x	15.5GiB
D_1^{full}	–	TO (0.31%)	–	–	26.8GiB
D_1'	300'000r	28.42min	108'372r/s	>6x	9.9GiB
D_1^{count}	32'399'833'908'509r	29.50min	104'425r/s	>6x	11.4GiB
Cyclic Query 7					
Scale Factor 1 (~1 GB)					
Baseline	239'917r	1.02min	124'981r/s	1.00x	8.5GiB
D_1^{full}	239'917r	1.44min	88'964r/s	0.71x	14.2GiB
D_1	239'917r	1.38min	92'548r/s	0.74x	12.9GiB
D_1^{count}	239'917r	1.60min	79'574r/s	0.64x	13.8GiB
D_1^{full}	239'917r	12.07min	10'601r/s	0.08x	31.4GiB
D_1'	239'917r	7.78min	16'514r/s	0.13x	35.8GiB
D_1^{count}	239'917r	15.37min	8312r/s	0.07x	32.7GiB
Scale Factor 10 (~10 GB)					
Baseline	2'398'579r	10.12min	126'328r/s	1.00x	10.9GiB
D_1^{full}	–	OOS (9.80%)	–	–	81.9GiB
D_1	2'398'579r	152.79min	8360r/s	0.07x	74.7GiB
D_1^{count}	–	OOS (8.07%)	–	–	75.7GiB
D_1^{full}	–	TO (2.33%)	–	–	90.0GiB
D_1'	–	TO (3.84%)	–	–	91.5GiB
D_1^{count}	–	TO (2.99%)	–	–	77.3GiB
Scale Factor 30 (~30 GB)					
Baseline	7'202'172r	30.29min	126'519r/s	1.00x	16.8GiB
D_1^{full}	–	TO (1.77%)	–	–	92.4GiB
D_1	–	TO (1.97%)	–	–	95.1GiB
D_1^{count}	–	TO (1.68%)	–	–	71.1GiB
D_1^{full}	–	TO (0.55%)	–	–	92.0GiB
D_1'	–	TO (0.78%)	–	–	100.4GiB
D_1^{count}	–	TO (0.53%)	–	–	91.0GiB

Table B.5: Performance comparison on the IMDB (JOB) dataset.
(~ 3.6 GB)

Variant	Output Size	Time	Throughput	Speedup	Peak RAM
<i>Acyclic Query 1</i>					
Baseline	702'744r	0.18min	359'041r/s	1.00x	2.6GiB
D_1^{full}	702'744r	0.18min	362'354r/s	1.00x	3.1GiB
D_1	3r	0.18min	359'097r/s	1.00x	2.5GiB
D_1^{count}	702'744r	0.18min	359'223r/s	1.00x	2.6GiB
$D_1'^{\text{full}}$	702'744r	0.19min	356'867r/s	0.95x	2.9GiB
D_1'	3r	0.19min	357'928r/s	0.95x	2.1GiB
$D_1'^{\text{count}}$	702'744r	0.19min	353'895r/s	0.95x	2.4GiB
D_2^{full}	702'744r	0.19min	355'113r/s	0.95x	2.8GiB
D_2	234'248r	0.19min	358'058r/s	0.95x	2.6GiB
D_2^{count}	702'744r	0.19min	357'109r/s	0.95x	2.7GiB
$D_2'^{\text{full}}$	702'744r	0.19min	357'238r/s	0.95x	3.1GiB
D_2'	234'248r	0.18min	359'189r/s	1.00x	2.5GiB
$D_2'^{\text{count}}$	702'744r	0.18min	358'757r/s	1.00x	2.6GiB
<i>Acyclic Query 2</i>					
Baseline	6'828'759r	2.91min	196'460r/s	1.00x	14.8GiB
D_1^{full}	6'828'759r	2.02min	282'854r/s	1.44x	15.1GiB
D_1	11r	1.18min	482'942r/s	2.47x	12.1GiB
D_1^{count}	6'828'759r	1.19min	481'355r/s	2.45x	12.6GiB
$D_1'^{\text{full}}$	6'828'759r	1.91min	298'888r/s	1.52x	14.4GiB
D_1'	11r	1.10min	517'528r/s	2.65x	10.8GiB
$D_1'^{\text{count}}$	6'828'759r	1.11min	517'182r/s	2.62x	11.4GiB
D_2^{full}	6'828'759r	2.27min	251'427r/s	1.28x	15.5GiB
D_2	161'149r	1.41min	406'286r/s	2.06x	12.6GiB
D_2^{count}	6'828'759r	1.46min	391'490r/s	1.99x	12.8GiB
$D_2'^{\text{full}}$	6'828'759r	2.11min	271'425r/s	1.38x	14.9GiB
D_2'	161'149r	1.19min	482'101r/s	2.45x	11.2GiB
$D_2'^{\text{count}}$	6'828'759r	1.27min	449'698r/s	2.29x	11.7GiB
<i>Cyclic Query 8</i>					
Baseline	1'916'563r	1.58min	283'665r/s	1.00x	10.9GiB
D_1^{full}	–	TO (63.28%)	–	–	11.8GiB
D_1	89'392r	1.11min	404'162r/s	1.42x	10.4GiB
D_1^{count}	1'916'563r	1.10min	404'840r/s	1.44x	10.6GiB
$D_1'^{\text{full}}$	–	TO (63.14%)	–	–	10.6GiB
D_1'	89'392r	1.22min	367'956r/s	1.30x	9.6GiB
$D_1'^{\text{count}}$	1'916'563r	1.58min	283'526r/s	1.00x	11.3GiB
D_2^{full}	1'916'563r	3.95min	113'210r/s	0.40x	11.5GiB
D_2	38'649r	3.80min	117'478r/s	0.42x	11.4GiB
D_2^{count}	1'916'563r	3.99min	111'826r/s	0.40x	11.9GiB
$D_2'^{\text{full}}$	1'916'563r	1.18min	377'105r/s	1.34x	10.7GiB
D_2'	38'649r	1.10min	404'627r/s	1.44x	9.6GiB
$D_2'^{\text{count}}$	1'916'563r	1.21min	368'281r/s	1.31x	10.0GiB

B.2 Ablation: terminal-view materialisation

Table B.6: Performance comparison on the **soc-Epinions1** dataset.
 $(|V| = 75'879, |E| = 508'837)$

Variant	Output Size	Time	Throughput	Speedup	Peak RAM
<i>Cyclic Query 3</i>					
Single Input Connector					
Baseline	100'317'892r	4.06min	2091r/s	1.00x	10.7GiB
D_1^{full}	100'317'892r	0.69min	12'330r/s	5.88x	9.6GiB
D_1'	3'548'205r	0.44min	19'439r/s	9.23x	6.1GiB
D_1^{count}	100'317'892r	0.46min	18'362r/s	8.83x	6.2GiB
5 Input Connectors					
Baseline	100'317'892r	4.14min	10'249r/s	1.00x	10.0GiB
D_1^{full}	100'317'892r	0.74min	57'375r/s	5.59x	10.1GiB
D_1'	3'548'205r	0.44min	96'869r/s	9.41x	6.3GiB
D_1^{count}	100'317'892r	0.44min	96'893r/s	9.41x	6.0GiB
<i>Cyclic Query 6</i>					
Single Input Connector					
Baseline	305'020'011r	1.50min	5676r/s	1.00x	13.7GiB
D_1^{full}	305'020'011r	2.27min	3732r/s	0.66x	13.3GiB
D_1'	30'482'554r	0.86min	9911r/s	1.74x	6.8GiB
D_1^{count}	305'020'011r	1.11min	7673r/s	1.35x	6.6GiB
4 Input Connectors					
Baseline	305'020'011r	1.58min	21'596r/s	1.00x	15.4GiB
D_1^{full}	305'020'011r	2.24min	15'129r/s	0.71x	13.3GiB
D_1'	30'482'554r	0.94min	36'167r/s	1.68x	7.2GiB
D_1^{count}	305'020'011r	1.11min	30'691r/s	1.42x	6.7GiB

Table B.7: Performance comparison on the **cit-HepTh** dataset.
 $(|V| = 27'770, |E| = 352'807)$

Variant	Output Size	Time	Throughput	Speedup	Peak RAM
<i>Cyclic Query 1</i>					
Single Input Connector					
Baseline	–	TO (22.82%)	–	–	49.0GiB
D_1^{full}	161'417'942r	0.94min	6273r/s	>191x	7.7GiB
D_1'	878'464r	0.35min	16'610r/s	>514x	5.0GiB
D_1^{count}	161'417'942r	0.35min	16'564r/s	>514x	5.0GiB
D_2^{full}	161'417'942r	1.02min	5758r/s	>176x	8.1GiB
D_2'	822'847r	0.44min	13'426r/s	>409x	6.0GiB
D_2^{count}	161'417'942r	0.60min	9744r/s	>300x	6.0GiB
9 Input Connectors					
Baseline	–	TO (22.82%)	–	–	49.0GiB
D_1^{full}	161'417'942r	1.13min	46'744r/s	>159x	8.6GiB
D_1'	878'464r	0.52min	101'601r/s	>346x	6.6GiB
D_1^{count}	161'417'942r	0.60min	87'658r/s	>300x	6.5GiB
D_2^{full}	161'417'942r	1.05min	50'495r/s	>171x	8.3GiB
D_2'	822'847r	0.52min	101'819r/s	>346x	6.4GiB
D_2^{count}	161'417'942r	0.60min	87'523r/s	>300x	6.2GiB
<i>Cyclic Query 2</i>					
Single Input Connector					
Baseline	266'404'327r	79.77min	74r/s	1.00x	16.3GiB
D_1^{full}	266'404'327r	1.91min	3077r/s	41.76x	14.6GiB

Variant	Output Size	Time	Throughput	Speedup	Peak RAM
D'_1	1'105'455r	0.44min	13'448r/s	181.30x	5.7GiB
D_1^{count}	266'404'327r	0.44min	13'443r/s	181.30x	5.3GiB
D_2^{full}	266'404'327r	1.52min	3863r/s	52.48x	10.4GiB
D'_2	2'023'391r	0.60min	9732r/s	132.95x	6.7GiB
D_2^{count}	266'404'327r	0.77min	7636r/s	103.60x	6.8GiB
8 Input Connectors					
Baseline	266'404'327r	77.75min	605r/s	1.00x	16.6GiB
D_1^{full}	266'404'327r	1.86min	25'349r/s	41.80x	15.2GiB
D'_1	1'105'455r	0.44min	107'503r/s	176.70x	6.3GiB
D_1^{count}	266'404'327r	0.49min	95'980r/s	158.67x	6.2GiB
D_2^{full}	266'404'327r	1.52min	30'882r/s	51.15x	10.2GiB
D'_2	2'023'391r	0.60min	77'953r/s	129.58x	6.9GiB
D_2^{count}	266'404'327r	0.77min	61'050r/s	100.97x	7.3GiB
<i>Cyclic Query 5</i>					
Single Input Connector					
Baseline	–	TO (71.21%)	–	–	18.7GiB
D_1^{full}	188'370'277r	1.86min	3168r/s	>96x	15.1GiB
D'_1	3'756'194r	1.02min	5750r/s	>176x	6.8GiB
D_1^{count}	188'370'277r	1.44min	4088r/s	>125x	7.7GiB
D_2^{full}	188'370'277r	1.41min	4170r/s	>127x	14.4GiB
D'_2	11'132'518r	0.60min	9723r/s	>300x	7.7GiB
D_2^{count}	188'370'277r	0.94min	6266r/s	>191x	7.8GiB
5 Input Connectors					
Baseline	–	TO (70.32%)	–	–	16.7GiB
D_1^{full}	188'370'277r	1.86min	15'847r/s	>96x	15.8GiB
D'_1	3'756'194r	1.02min	28'767r/s	>176x	7.2GiB
D_1^{count}	188'370'277r	1.47min	20'053r/s	>122x	7.9GiB
D_2^{full}	188'370'277r	1.44min	20'421r/s	>125x	14.9GiB
D'_2	11'132'518r	0.60min	48'751r/s	>300x	7.6GiB
D_2^{count}	188'370'277r	0.94min	31'359r/s	>191x	7.5GiB

Table B.8: Performance comparison on the p2p-Gnutella31 dataset.
 $(|V| = 62'586, |E| = 147'892)$

Variant	Output Size	Time	Throughput	Speedup	Peak RAM
<i>5 Path Query</i>					
Single Input Connector					
Baseline	26'632'138r	0.19min	13'130r/s	1.00x	6.0GiB
D_1^{full}	26'632'138r	0.19min	13'174r/s	1.00x	6.1GiB
D'_1	143'932r	0.11min	23'454r/s	1.73x	0.4GiB
D_1^{count}	26'632'138r	0.10min	24'234r/s	1.90x	0.4GiB
D_2^{full}	26'632'138r	0.13min	20'135r/s	1.46x	6.1GiB
D'_2	52'046r	0.10min	23'838r/s	1.90x	0.4GiB
D_2^{count}	26'632'138r	0.10min	24'270r/s	1.90x	0.4GiB
5 Input Connectors					
Baseline	26'632'138r	0.19min	65'759r/s	1.00x	6.0GiB
D_1^{full}	26'632'138r	0.19min	65'468r/s	1.00x	6.2GiB
D'_1	143'932r	0.10min	120'085r/s	1.90x	0.5GiB
D_1^{count}	26'632'138r	0.10min	118'033r/s	1.90x	0.6GiB
D_2^{full}	26'632'138r	0.19min	66'389r/s	1.00x	6.1GiB
D'_2	52'046r	0.10min	118'447r/s	1.90x	0.5GiB
D_2^{count}	26'632'138r	0.10min	120'230r/s	1.90x	0.6GiB

Variant	Output Size	Time	Throughput	Speedup	Peak RAM
<i>Cyclic Query 4</i>					
Single Input Connector					
Baseline	16'058r	0.10min	23'836r/s	1.00x	2.4GiB
D_1^{full}	16'058r	0.33min	7683r/s	0.30x	8.0GiB
D_1'	9672r	0.27min	9077r/s	0.37x	5.7GiB
D_1^{count}	16'058r	0.30min	8363r/s	0.33x	6.2GiB
D_2^{full}	16'058r	0.19min	13'289r/s	0.53x	3.4GiB
D_2'	10'268r	0.19min	13'113r/s	0.53x	2.9GiB
D_2^{count}	16'058r	0.19min	13'090r/s	0.53x	3.4GiB
6 Input Connectors					
Baseline	16'058r	0.10min	143'262r/s	1.00x	2.5GiB
D_1^{full}	16'058r	0.30min	50'222r/s	0.33x	8.0GiB
D_1'	9672r	0.27min	54'848r/s	0.37x	5.6GiB
D_1^{count}	16'058r	0.27min	54'887r/s	0.37x	6.5GiB
D_2^{full}	16'058r	0.19min	79'541r/s	0.53x	3.7GiB
D_2'	10'268r	0.19min	78'474r/s	0.53x	3.0GiB
D_2^{count}	16'058r	0.19min	78'620r/s	0.53x	3.7GiB

Table B.9: Performance comparison on the TPC-H dataset.

Variant	Output Size	Time	Throughput	Speedup	Peak RAM
<i>4 Table Path Query</i>					
Scale Factor 1 (~ 1 GB)					
Baseline	6'001'215r	1.02min	137'691r/s	1.00x	9.5GiB
D_1^{full}	6'001'215r	1.05min	134'648r/s	0.97x	14.3GiB
D_1	6'001'215r	1.02min	137'855r/s	1.00x	10.7GiB
D_1^{count}	6'001'215r	1.02min	137'993r/s	1.00x	10.9GiB
D_1^{full}	6'001'215r	1.02min	137'962r/s	1.00x	11.2GiB
D_1'	6'001'215r	1.02min	137'996r/s	1.00x	9.3GiB
D_1^{count}	6'001'215r	1.02min	137'969r/s	1.00x	9.3GiB
Scale Factor 10 (~ 10 GB)					
Baseline	59'986'052r	10.17min	138'603r/s	1.00x	18.2GiB
D_1^{full}	59'986'052r	11.56min	121'840r/s	0.88x	37.7GiB
D_1	59'986'052r	10.15min	138'795r/s	1.00x	23.2GiB
D_1^{count}	59'986'052r	10.25min	137'334r/s	0.99x	18.5GiB
D_1^{full}	59'986'052r	10.73min	131'264r/s	0.95x	29.1GiB
D_1'	59'986'052r	9.84min	143'123r/s	1.03x	12.2GiB
D_1^{count}	59'986'052r	9.81min	143'521r/s	1.04x	11.2GiB
Scale Factor 30 (~ 30 GB)					
Baseline	179'998'372r	34.85min	121'247r/s	1.00x	24.3GiB
D_1^{full}	179'998'372r	50.20min	84'179r/s	0.69x	43.0GiB
D_1	179'998'372r	33.79min	125'035r/s	1.03x	33.5GiB
D_1^{count}	179'998'372r	31.15min	135'699r/s	1.12x	28.3GiB
D_1^{full}	179'998'372r	39.33min	107'448r/s	0.89x	41.7GiB
D_1'	179'998'372r	29.62min	142'653r/s	1.18x	14.8GiB
D_1^{count}	179'998'372r	30.42min	138'910r/s	1.15x	13.5GiB
<i>5 Table Path Query</i>					
Scale Factor 1 (~ 1 GB)					
Baseline	6'001'215r	1.02min	124'867r/s	1.00x	9.8GiB
D_1^{full}	6'001'215r	1.05min	121'927r/s	0.97x	16.8GiB
D_1	6'001'215r	1.02min	125'256r/s	1.00x	11.4GiB

Variant	Output Size	Time	Throughput	Speedup	Peak RAM
D_1^{count}	6'001'215r	1.02min	125'303r/s	1.00x	11.5GiB
D_1^{full}	6'001'215r	1.08min	118'907r/s	0.94x	13.0GiB
D_1'	6'001'215r	0.99min	128'657r/s	1.03x	9.4GiB
D_1^{count}	6'001'215r	1.02min	125'105r/s	1.00x	9.7GiB
Scale Factor 10 (~ 10 GB)					
Baseline	59'986'052r	10.70min	119'427r/s	1.00x	24.9GiB
D_1^{full}	59'986'052r	12.98min	98'605r/s	0.82x	44.0GiB
D_1	59'986'052r	10.17min	125'470r/s	1.05x	23.1GiB
D_1^{count}	59'986'052r	10.20min	125'139r/s	1.05x	20.6GiB
D_1^{full}	59'986'052r	12.12min	105'323r/s	0.88x	32.4GiB
D_1'	59'986'052r	9.87min	129'401r/s	1.08x	12.2GiB
D_1^{count}	59'986'052r	9.87min	129'378r/s	1.08x	12.1GiB
Scale Factor 30 (~ 30 GB)					
Baseline	179'998'372r	39.69min	96'681r/s	1.00x	25.8GiB
D_1^{full}	179'998'372r	54.60min	70'167r/s	0.73x	49.1GiB
D_1	179'998'372r	32.26min	118'718r/s	1.23x	27.4GiB
D_1^{count}	179'998'372r	31.56min	121'477r/s	1.26x	26.4GiB
D_1^{full}	179'998'372r	49.56min	77'296r/s	0.80x	40.3GiB
D_1'	179'998'372r	29.59min	129'428r/s	1.34x	15.9GiB
D_1^{count}	179'998'372r	30.48min	125'718r/s	1.30x	13.5GiB
3 Table Path Query					
Scale Factor 1 (~ 1 GB)					
Baseline	–	TO (5.35%)	–	–	23.8GiB
D_1^{full}	–	TO (5.80%)	–	–	23.8GiB
D_1	10'000r	0.96min	106'610r/s	>187x	7.6GiB
D_1^{count}	36'007'310'944r	1.02min	100'755r/s	>176x	8.0GiB
D_1^{full}	–	TO (5.75%)	–	–	23.9GiB
D_1'	10'000r	0.94min	109'421r/s	>191x	6.5GiB
D_1^{count}	36'007'310'944r	0.94min	109'501r/s	>191x	6.8GiB
Cyclic Query 7					
Scale Factor 1 (~ 1 GB)					
Baseline	239'917r	1.02min	124'839r/s	1.00x	8.5GiB
D_1^{full}	239'917r	1.41min	90'766r/s	0.72x	13.2GiB
D_1	239'917r	1.32min	96'500r/s	0.77x	11.4GiB
D_1^{count}	239'917r	1.44min	88'884r/s	0.71x	12.0GiB
D_1^{full}	239'917r	9.73min	13'268r/s	0.10x	25.5GiB
D_1'	239'917r	5.83min	21'898r/s	0.17x	17.9GiB
D_1^{count}	239'917r	9.84min	12'978r/s	0.10x	20.2GiB

Table B.10: Performance comparison on the IMDB (JOB) dataset.
(~ 3.6 GB)

Variant	Output Size	Time	Throughput	Speedup	Peak RAM
Acyclic Query 2					
Baseline	6'828'759r	2.88min	198'336r/s	1.00x	14.6GiB
D_1^{full}	6'828'759r	2.02min	282'645r/s	1.43x	15.0GiB
D_1	11r	1.19min	481'509r/s	2.42x	12.0GiB
D_1^{count}	6'828'759r	1.19min	481'686r/s	2.42x	12.2GiB
D_1^{full}	6'828'759r	1.85min	308'342r/s	1.56x	13.7GiB
D_1'	11r	1.10min	518'779r/s	2.62x	10.5GiB
D_1^{count}	6'828'759r	1.10min	518'669r/s	2.62x	10.6GiB
D_2^{full}	6'828'759r	2.27min	251'483r/s	1.27x	16.0GiB

Variant	Output Size	Time	Throughput	Speedup	Peak RAM
D_2	161'149r	1.35min	422'161r/s	2.13x	12.4GiB
D_2^{count}	6'828'759r	1.44min	397'519r/s	2.00x	12.5GiB
D_2^{full}	6'828'759r	2.05min	278'572r/s	1.40x	14.6GiB
D_2'	161'149r	1.19min	481'953r/s	2.42x	10.6GiB
$D_2'^{\text{count}}$	6'828'759r	1.22min	470'985r/s	2.36x	10.6GiB
<i>Cyclic Query 8</i>					
Baseline	1'916'563r	1.52min	293'010r/s	1.00x	10.8GiB
D_1^{full}	–	TO (63.28%)	–	–	10.9GiB
D_1	89'392r	1.10min	404'478r/s	1.38x	10.1GiB
D_1^{count}	1'916'563r	1.10min	404'711r/s	1.38x	10.3GiB
D_1^{full}	–	TO (63.16%)	–	–	10.0GiB
D_1'	89'392r	1.19min	376'648r/s	1.28x	8.4GiB
$D_1'^{\text{count}}$	1'916'563r	1.44min	310'563r/s	1.06x	9.6GiB
D_2^{full}	1'916'563r	3.94min	113'295r/s	0.39x	11.2GiB
D_2	38'649r	3.77min	118'336r/s	0.40x	10.9GiB
D_2^{count}	1'916'563r	3.83min	116'573r/s	0.40x	11.5GiB
D_2^{full}	1'916'563r	1.13min	395'012r/s	1.35x	9.9GiB
D_2'	38'649r	1.10min	404'598r/s	1.38x	8.8GiB
$D_2'^{\text{count}}$	1'916'563r	1.19min	376'436r/s	1.28x	9.2GiB

Bibliography

- [1] Serge Abiteboul, Richard Hull, and Victor Vianu. *Foundations of Databases*. Addison-Wesley, 1995.
- [2] Mahmoud Abo Khamis, Ahmet Kara, Dan Olteanu, and Dan Suciu. Insert-Only versus Insert-Delete in Dynamic Query Evaluation. *Proc. ACM Manag. Data*, 2(5):219:1–219:26, 2024.
- [3] Albert Atserias, Martin Grohe, and Dániel Marx. Size Bounds and Query Plans for Relational Joins. *SIAM J. Comput.*, 42(4):1737–1767, 2013.
- [4] Hans L. Bodlaender. Discovering Treewidth. In Peter Vojtás, Mária Bieliková, Bernadette Charron-Bost, and Ondrej Sýkora, editors, *SOFSEM 2005: Theory and Practice of Computer Science, 31st Conference on Current Trends in Theory and Practice of Computer Science, Liptovský Ján, Slovakia, January 22–28, 2005, Proceedings*, volume 3381 of *Lecture Notes in Computer Science*, pages 1–16. Springer, 2005.
- [5] Mihai Budiu, Tej Chajed, Frank McSherry, Leonid Ryzhyk, and Val Tannen. DBSP: Automatic Incremental View Maintenance for Rich Query Languages. *Proc. VLDB Endow.*, 16(7):1601–1614, 2023.
- [6] Marek Cygan, Fedor V. Fomin, Lukasz Kowalik, Daniel Lokshantov, Dániel Marx, Marcin Pilipczuk, Michal Pilipczuk, and Saket Saurabh. *Parameterized Algorithms*. Springer, 2015.
- [7] Feldera, Inc. Feldera. <https://www.feldera.com>, 2026. Accessed: 2026-07-15.
- [8] Wolfgang Fischl, Georg Gottlob, and Reinhard Pichler. General and Fractional Hypertree Decompositions: Hard and Easy Cases. In Jan Van den Bussche and Marcelo Arenas, editors, *Proceedings of the 37th ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems, Houston, TX, USA, June 10–15, 2018*, pages 17–32. ACM, 2018.
- [9] Todd J. Green, Gregory Karvounarakis, and Val Tannen. Provenance semirings. In Leonid Libkin, editor, *Proceedings of the Twenty-Sixth ACM SIGACT-SIGMOD-SIGART Symposium on Principles of Database Systems, June 11–13, 2007, Beijing, China*, pages 31–40. ACM, 2007.
- [10] Martin Grohe and Dániel Marx. Constraint Solving via Fractional Edge Covers. *ACM Trans. Algorithms*, 11(1):4:1–4:20, 2014.
- [11] Viktor Leis, Andrey Gubichev, Atanas Mirchev, Peter Boncz, Alfons Kemper, and Thomas Neumann. How Good Are Query Optimizers, Really? *Proc. VLDB Endow.*, 9(3):204–215, 2015.
- [12] Jure Leskovec and Andrej Krevl. SNAP Datasets: Stanford Large Network Dataset Collection. <http://snap.stanford.edu/data>, June 2014.
- [13] Dániel Marx. Approximating fractional hypertree width. *ACM Trans. Algorithms*, 6(2):29:1–29:17, 2010.
- [14] Dan Olteanu. Recent Increments in Incremental View Maintenance. In Floris Geerts and Wim Martens, editors, *Companion of the 43rd Symposium on Principles of Database Systems, PODS 2024, Santiago, Chile, June 9–15, 2024*, pages 8–17. ACM, 2024.
- [15] Meikel Pöss and Chris Floyd. New TPC Benchmarks for Decision Support and Web Commerce. *SIGMOD Rec.*, 29(4):64–71, 2000.

Declaration on the Use of AI Tools

AI tools were used during the drafting of this thesis strictly for text polishing, rephrasing, and grammatical editing.